From Specification Violations to Exploitability: Deviation-Guided Testing of TLS Libraries

***Abstract*—Transport Layer Security (TLS) libraries implement stateful handshakes in which message acceptance depends on protocol state and context. A TLS library may therefore accept a specification-violating message without crashing or failing visibly. Such bugs are hard to find without a specification-level oracle—and even once found, judging exploitability is a separate, largely manual task. We address both detection and exploitability by making off-specification behavior and exploitable outcomes explicit targets for test generation. At its core is an executable formal model of the TLS handshake, built from the TLS RFCs, that serves as a specification-level oracle. Over it we define *behavior deviations*, parameterized ways a handshake can depart from the specification, and *scenario properties*, which steer generation toward outcomes from rejection through acceptance to completion under an invalid message. Both are reusable across handshake contexts and extensible for target-specific behavior. Our framework compiles these into black-box tests against unmodified TLS libraries and continues each exposed violation on the same library—automatically when the violated requirement is modeled—until a completed handshake is reached, exposing the security impact of the accepted behavior, such as peer impersonation or connection downgrade. Within the modeled TLS handshake scope, we find 16 previously unknown specification violations—confirmed by developers—in recent stable versions of GnuTLS, MbedTLS, and WolfSSL, of which 8 were assigned new CVEs; the framework also finds substantially more RFC-level violations than prior state-of-the-art TLS testing tools.**

I. Introduction

Transport Layer Security (TLS) secures most network communication, from web browsing and email to APIs and mobile applications. Its security rests on a *handshake*: a stateful exchange in which a client and server negotiate cryptographic parameters, authenticate each other, and establish keys that protect subsequent communication. The handshake is defined by the TLS RFCs [1], [2], whose design drew on extensive formal analysis [3], [4]. Because every TLS session begins with this handshake, flaws in its implementations can undermine authentication, weaken confidentiality, or compromise forward secrecy for every connection they serve. Many real-world attacks have exploited such handshake flaws, making both *detection* of handshake bugs and assessment of *whether they are exploitable* central to the security of deployed systems.

Many important handshake bugs are not crashes but *silent acceptances*: a library accepts a message that the RFC requires it to reject and continues the handshake instead of aborting. Whether a message is acceptable depends on protocol state and negotiated context, so the same message can be acceptable in one context and forbidden in another. Such bugs produce no crash and no visible failure, so techniques that look for memory errors or observable misbehavior do not detect them. Detecting them instead requires a *specification-level oracle*: a precise, RFC-grounded account of which messages an endpoint may accept in each context.

Existing TLS testing (e.g., [5]–[8]) has made substantial progress in exposing conformance and implementation bugs. However, it separates the two halves of the problem: violations are *detected* against one representation, while *exploitability*—whether a detected violation can be driven to a security-relevant outcome—is assessed, if at all, by separate and manual analysis. ProInspector [9] is the closest existing approach, but it exemplifies this separation: its exploitability analysis runs over a symbolic, model-level representation rather than against the deployed library that exhibited the violation. More generally, existing formal TLS models target protocol-level security analysis and abstract away the implementation-level details that RFC-conformance testing needs—concrete negotiated values, message-processing checks, and executable error behavior. Detection and exploitability thus remain decoupled, with no unified way to carry implementation-level violations to implementation-level security outcomes.

Addressing this problem requires three capabilities that existing techniques do not jointly provide. (1) An *executable, RFC-precise* oracle that decides, message by message, whether an endpoint may accept a message in its negotiated context—precise enough to distinguish a specification violation from acceptable behavior in the same protocol state. (2) A way to express *off-specification* behavior that is parameterized and tied to the RFC requirement it violates, rather than arbitrary mutation that obscures which requirement was broken and cannot reliably steer a handshake into the states where violations occur. (3) A way to carry a detected violation through to a security-relevant outcome *on the real library*: acceptance of an invalid message is only an intermediate signal, and its impact is established only by continuing the handshake, against the unmodified implementation, until the consequence is realized.

We address this by making off-specification behavior and exploitable outcomes *explicit targets for test generation* over a single precise oracle. At its core is an executable formal model of the TLS 1.2 and 1.3 handshake, built from RFC 5246 and RFC 8446, that decides which messages an endpoint may accept in each negotiated context. Over this oracle we define two constructs. *Behavior deviations* describe, in parameterized form, how a handshake may depart from the specification—a malformed or omitted message, a bypassed validation check, or an invalid negotiated state—in terms of the RFC requirements they violate rather than as arbitrary mutations. *Scenario properties* steer test generation toward a

chosen outcome: rejection, invalid acceptance, or completion of the handshake under an invalid message. Because both constructs share the same model, detection and exploitability are expressed in the same terms: a violation and its security consequence are reasoned about against a single RFC-level account of the protocol, not across separate artifacts.

Our framework compiles these constructs into black-box tests against unmodified TLS libraries. When a test exposes an invalid acceptance, the framework continues the handshake to completion on the same library, so exploitability is established *at the implementation level* rather than inferred from the model. Because the model captures the message-acceptance logic the tests exercise, violations represented in the model can be carried to implementation-level outcomes automatically. For example, the framework exercises the full workflow on a TLS 1.3 resumption flaw (Section VII-C): a session resumed from a pre-shared key preserves forward secrecy only when combined with a fresh key exchange. A behavior deviation expresses the acceptance of a resumption message that should have been rejected, a scenario property drives the session to completion, and the continued handshake, run against the unmodified library, completes a resumed session without forward secrecy—an exploitable vulnerability on the deployed implementation, not merely in the model.

On recent stable versions of the widely deployed open-source TLS libraries GnuTLS, MbedTLS, and WolfSSL, our evaluation reports 16 previously unknown specification violations, all confirmed by developers. For silent acceptances, exploitability analysis continues the accepted invalid behavior on the unmodified library until a completed handshake is reached, exposing the security impact of the violation on the deployed code. This analysis contributed to assessing the severity of these findings; in total, 8 of the reported violations were assigned new CVEs. Against three state-of-the-art TLS testing tools [10]–[12] on the same library versions, our approach consistently finds more RFC-level violations; e.g., in the TLS-Anvil settings, it finds 82 violations, compared with 23 reported by TLS-Anvil. We also reproduce 9 previously known TLS CVEs from deviation-guided scenarios, showing that the approach can re-derive established attack patterns.

In summary, our main contributions are as follows:

(1) An *executable, RFC-precise formal model* of the TLS 1.2/1.3 handshake that serves as an oracle for endpoint message acceptance in negotiated contexts.
(2) *Behavior deviations* for parameterized off-specification behaviors, and *scenario properties* that steer test generation toward rejection, invalid acceptance, or exploitability.
(3) A *deviation-guided testing framework* that generates black-box tests for unmodified TLS libraries and carries violations into implementation-level exploitability.
(4) An *evaluation* showing that the framework finds more RFC-level violations than prior tools on the same library versions, with 8 new CVE-assigned vulnerabilities.

Our artifact [13] provides the formal model, the benchmark, and test-generation scripts for reproducing the experiments, along with details on the model, test specifications, and results.

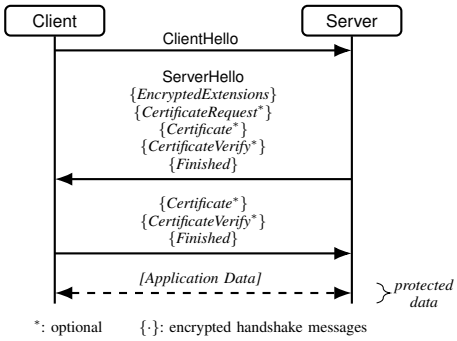

Fig. 1. Representative ordered message flow of a TLS 1.3 full handshake.

II. Background

TLS Protocol: TLS, as specified by RFCs 5246 and 8446 [1], [2], is a stateful protocol in which client and server endpoints exchange handshake messages to negotiate protocol parameters, establish the authentication context, and derive keys for protecting subsequent application data. As illustrated by the representative TLS 1.3 flow in Figure 1, a handshake is an ordered message sequence. At the message-processing level, we distinguish three kinds of endpoint behavior. First, an endpoint constructs an outgoing handshake message from its local state and sends it. Second, the peer receives that message, checks it against its current local state and the TLS RFC requirements, and updates its state when the checks succeed. Third, if the message is invalid in that state, the peer sends an alert instead of continuing the handshake.

For example, a TLS 1.3 `ClientHello` advertises the client's supported cipher suites and a `key_share` for ephemeral key establishment. The server checks these against its state and the RFC requirements, selects compatible parameters, and stores them in its local state.

A handshake is called *completed* when both endpoints exchange and verify `Finished` messages, enabling application data exchange; a handshake that terminates earlier, whether by alert or by silent termination, does not complete.

Threat Model: We adopt a Dolev–Yao-style adversary at the TLS-message level [14], as commonly used to analyze protocol designs [3], [4], [15] and implementation behavior [10]. The adversary either controls one TLS endpoint that communicates with a target implementation or acts as a man-in-the-middle in the network, and can send malformed TLS messages, omit expected messages, replay or reorder modeled messages, and observe the target's responses. It may compute values using keys and secrets it knows, but is not assumed to produce ciphertexts, signatures, or other data that depend on unknown secrets, and it does not modify the target source code, binary files, or runtime memory.

III. Motivating Example

We use TLS 1.3 pre-shared key (PSK) resumption to illustrate the challenges of detecting TLS specification violations and assessing their exploitability. A client can use TLS 1.3 resumption to establish a new connection with a PSK derived

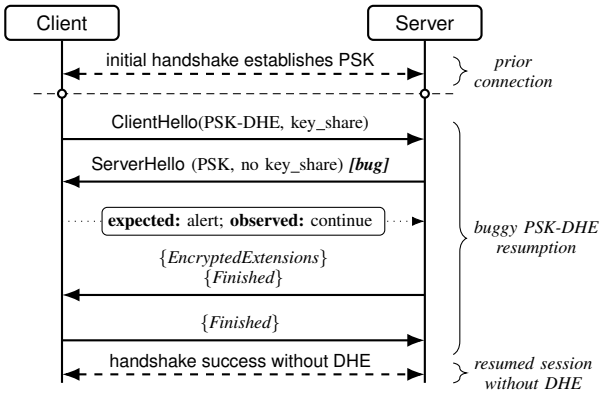

Fig. 2. TLS 1.3 PSK-DHE resumption bug

from an earlier connection, either in PSK-only mode or in PSK-DHE mode; PSK-DHE is the forward-secret mode: it adds an ephemeral key exchange, so the resumed session does not rely only on the reused PSK. RFC 8446 states:

> "`psk_dhe_ke`: In this mode, client and server **MUST** supply `key_share` values" (RFC 8446 [2], Sec. 4.2.9).

The RFC-level violation arises in a PSK-DHE resumption attempt (Figure 2). A server sends a `ServerHello` that selects the PSK but omits `key_share`; because the client requested PSK-DHE, the response is context-invalid and should be rejected. However, a buggy client accepts the invalid response without raising an error and proceeds in PSK-only mode.

The server completes the remaining flight, including a valid `Finished`; the client thus completes a handshake without the requested forward-secret mode, risking decryption of recorded traffic if the reused secret is later compromised [16]. This buggy behavior occurs in WolfSSL [17], one of the CVEs we reported and confirm as a new vulnerability (see Table IV).

Finding this bug requires more than exercising TLS resumption. The analysis must turn precise RFC requirements in a resumed-handshake context into a negative scenario, not merely a malformed message or generic reconnect workflow. Accepting the invalid `ServerHello` is still only an intermediate signal: to expose the security-relevant outcome, the same execution must continue through an RFC-consistent handshake path with messages compatible with the PSK context and `Finished` validation. Otherwise, the run may fail before revealing that the library can complete a resumed session.

This example shows why existing TLS testing tools are insufficient for this kind of analysis: workflow-based testers exercise resumption but cannot express this context-specific negative case, and message-level fuzzers that could flag the omission still need a way to carry it to a completed handshake to expose the impact. The analysis needs an executable, precise TLS model as a specification oracle, a way to express context-specific modifications such as removing `key_share`, and a scenario property describing the sequence from PSK establishment to a downgraded, completed resumption. We therefore propose a formal TLS model, behavior deviations, scenario properties, and a framework that integrates them into concrete tests against TLS libraries in the rest of this paper.

IV. Formal Specification of TLS using Maude

This section defines an executable formal model of TLS handshake behavior as a specification-level oracle. We develop the model in Maude [18], [19], a mature formal specification and analysis framework widely used for concurrent systems [20]. Its object-oriented specifications naturally represent protocol participants, messages, and adversarial network behavior as algebraic terms and object configurations. Together with Maude's search, model-checking, and strategy-based exploration capabilities, this representation supports well automated exploration of TLS behaviors for test generation.

A. TLS System Specification using Maude

We model a TLS handshake as a concurrent object system: client and server objects, network links, and conditional rewrite rules encode message-processing behavior. These rules fall into three types: send rules construct handshake messages, receive rules apply RFC checks and update endpoint state, and error rules model alerts for invalid inputs.

In a TLS object configuration, client and server objects store endpoint-local TLS state, and two directional link objects store in-transit messages as ordered queues. Endpoint-local state includes supported and selected parameters, and runtime metadata for message construction and validation. We write an object as $\langle o : C \mid a_1 : v_1, \ldots, a_n : v_n \rangle$, with identifier o, class C, and attribute-value pairs $a_i : v_i$.

The below code shows an initial Maude TLS object configuration. The objects `ci` and `si` denote the client and server. Their `status` values put the client in the `ClientHello`-sending state and the server in the corresponding receiving state. `keyShareGroup` stores supported key-exchange groups; `noGroup` specify the absence of a negotiated group.

```
< ci : Client | status : cinit, selectedGroup : noGroup,
keyShareGroup : secp256r1 secp384r1, nonceCtr : 0, ... >
< si : Server | status : sready, selectedGroup : noGroup,
keyShareGroup : secp256r1, nonceCtr : 0, ... >
< ci to si : Link | msgQueue : empty >
< si to ci : Link | msgQueue : empty > .
```

We introduce three rewrite-rule patterns for message processing, each written as $[\ell]\colon L \Rightarrow R$ `if` C, where the label ℓ names the transition, L matches the relevant objects in the current configuration, and R gives the updated objects when the condition C is satisfied. The send pattern matches the outgoing link from o_{src} to o_{dst} and the source endpoint o_{src}. Here, `MSGS` is the outgoing queue, st is the matched control state, and v_i are the matched attribute values. The condition constructs `MSG` as fields $f_i(m_i)$, where f_i names a message field and m_i is an expression over the matched values, such as $opr(v_1, \ldots, v_n)$. On the right-hand side, `MSG` is appended to the queue, the control state advances to st', and the attributes a_i are updated to values v_i'.

$$
\begin{aligned}
&[\texttt{send}\mathit{MsgType}]\colon \\
&\quad \langle o_{src} \texttt{ to } o_{dst} : \texttt{Link} \mid \texttt{msgQueue} : \texttt{MSGS} \rangle \\
&\quad \langle o_{src} : \mathit{NodeType} \mid \texttt{status} : st, a_1 : v_1, \ldots, a_n : v_n \rangle \\
&\Rightarrow \langle o_{src} \texttt{ to } o_{dst} : \texttt{Link} \mid \texttt{msgQueue} : \texttt{MSGS} :: \texttt{MSG} \rangle \\
&\quad \langle o_{src} : \mathit{NodeType} \mid \texttt{status} : st', a_1 : v_1', \ldots, a_n : v_n' \rangle \\
&\texttt{if MSG} := f_1(m_1) \cdots f_n(m_n) \ .
\end{aligned}
$$

```
crl [sendClientHello]:
   < ci to si : Link | msgQueue : MSGS >
   < ci : Client | status : cinit, nonceCtr : N,
     keyShareGroup : KGL, ... >
=> < ci to si : Link | msgQueue : MSGS :: MSG >
   < ci : Client | status : chello,nonceCtr : N+|KGL|, ... >
if MSG := mType(clienthello)... keyShare(genEntry(KGL, N)) .

crl [receiveClientHello]:
   < ci to si : Link | msgQueue : MSG :: MSGS >
   < si : Server | status : sready, keyShareGroup : KGL,
     nonceCtr : N, selectedGroup : noGroup, ... >
=> < ci to si : Link | msgQueue : MSGS >
   < si : Server | status : shello,
     selectedGroup : select(KGL, MSG), ... >
if checkRFC({keyShare-exists, ...},MSG,< si : Server | >) .

crl [errorClientHello]:
   < ci to si : Link | msgQueue : MSG :: MSGS >
   < si : Server | status : sready, ... >
   < si to ci : Link | msgQueue : MSGS' >
=> < ci to si : Link | msgQueue : empty >
   < si : Server | status : error >
   < si to ci : Link | msgQueue : MSGS' :: ALRT >
if not checkRFC({keyShare-exists,...},MSG,< si : Server | >)
/\ ALRT := mType(alert) alrtDesc(MSG, < si : Server | >) .
```

Fig. 3. Maude object configuration and rewrite rules for TLS handshake.

The `sendClientHello` rule in Figure 3 instantiates the send pattern for the initial `ClientHello`. The client uses the key-share groups `KGL` and counter `N` to generate the message, appends it to the outgoing queue, and moves to status `chello`.

The receive pattern matches the incoming link and endpoint o_{dst}, with `MSG` at the head of the queue. Its condition evaluates the RFC requirements Λ in the current context. If validated, the rule removes `MSG`, advances the status to st', and updates the attributes to v_i'. In the `receiveClientHello` rule, the server checks the message, including `key_share` presence. If the checks pass, the server consumes the message, stores the selected key-share group, and moves to status `shello`.

$$
\begin{aligned}
&[\texttt{receive}\mathit{MsgType}]\texttt{:}\\
&\quad \langle o_{src}\ \texttt{to}\ o_{dst} : \texttt{Link} \mid \texttt{msgQueue} : \texttt{MSG} :: \texttt{MSGS}\rangle\\
&\quad \langle o_{dst} : \mathit{NodeType} \mid \texttt{status} : st, a_1 : v_1, \ldots, a_n : v_n\rangle\\
&\Rightarrow \langle o_{src}\ \texttt{to}\ o_{dst} : \texttt{Link} \mid \texttt{msgQueue} : \texttt{MSGS}\rangle\\
&\quad \langle o_{dst} : \mathit{NodeType} \mid \texttt{status} : st', a_1 : v_1', \ldots, a_n : v_n'\rangle\\
&\texttt{if checkRFC}(\Lambda,\ \texttt{MSG},\ \langle o_{dst} : \mathit{NodeType} \mid \rangle)\ .
\end{aligned}
$$

The error pattern handles failed receive-side RFC checks. It negates `checkRFC`, constructs `ALERT`, clears the incoming queue, moves the endpoint to `error`, and appends the alert to the reverse queue. The `errorClientHello` rule instantiates this failure case for `ClientHello`.

$$
\begin{aligned}
&[\texttt{error}\mathit{MsgType}]\texttt{:}\\
&\quad \langle o_{src}\ \texttt{to}\ o_{dst} : \texttt{Link} \mid \texttt{msgQueue} : \texttt{MSG} :: \texttt{MSGS}\rangle\\
&\quad \langle o_{dst} : \mathit{NodeType} \mid \texttt{status} : st,\ a_1 : v_1, \ldots, a_n : v_n\rangle\\
&\quad \langle o_{dst}\ \texttt{to}\ o_{src} : \texttt{Link} \mid \texttt{msgQueue} : \texttt{MSGS'}\rangle\\
&\Rightarrow \langle o_{src}\ \texttt{to}\ o_{dst} : \texttt{Link} \mid \texttt{msgQueue} : \texttt{empty}\rangle\\
&\quad \langle o_{dst} : \mathit{NodeType} \mid \texttt{status} : \texttt{error}, \ldots\rangle\\
&\quad \langle o_{dst}\ \texttt{to}\ o_{src} : \texttt{Link} \mid \texttt{msgQueue} : \texttt{MSGS'} :: \texttt{ALRT}\rangle\\
&\texttt{if}\ \neg\ \texttt{checkRFC}(\Lambda,\ \texttt{MSG},\ \langle o_{dst} : \mathit{NodeType} \mid \rangle)\\
&\quad \wedge\ \texttt{ALRT} := f_1(m_1)\cdots f_n(m_n)\ .
\end{aligned}
$$

The Maude model can be viewed as a labeled transition system whose states are TLS object configurations. An action denotes one rewrite-rule application: whether it sends, receives, or rejects a message, the message type, and the matched objects. The transition relation connects two states with such an action.

TABLE I
COMPARISON OF MODELED TLS FEATURES.

TLS Feature		ProVerif [4]	Tamarin [3]	This Work
Negotiation	VER	X	X	O
	CS	X	X	O
	HRR	△	△	O
	EXT	△	△	O
Authentication	SRV	O	O	O
	CLI	O	O	O
	PHA	O	O	O
Pre-shared keys	PSK-DHE	O	O	O
	TKT	O	O	O
	PSKO	X	O	O
	0RTT	O	O	O
Key Update	KUP	X	O	O

B. Modeling Scope

The effectiveness of model-based RFC-conformance testing depends on how much of the relevant RFC message-processing behavior the model's oracle encodes. Our model specifies RFC-level TLS 1.2 and TLS 1.3 handshake behaviors, including negotiation, authentication, PSK-based resumption, 0-RTT, and KeyUpdate. For each TLS feature, our model encodes the requirements that an endpoint must satisfy when sending or receiving messages. We model MUST and MUST NOT statements across the TLS 1.2 and TLS 1.3 specifications and report coverage by specification section in [13].

Concrete test-case generation requires the model to represent the values that determine TLS negotiation, not only message order or fixed templates. Endpoint objects therefore store values needed to construct and validate messages, including supported and selected parameters, resumption context. Each modeled TLS message records the fields inspected by RFC requirements and their assigned values, rather than only its handshake-message type. For example, a `key_share` field pairs a key-exchange group with the nonce value.

Table I compares our model with representative symbolic TLS models by the TLS features they encode. The table schema follows ProVerif's comparison table [4, Table 1], but adds generic extension handling and post-handshake KeyUpdate. Extension handling matters because many TLS 1.3 handshake parameters are negotiated through extensions, making it central to concrete message-processing checks. In the table, O denotes explicit modeling, △ denotes support only under fixed parameter choices, and X denotes features not modeled.

The comparison shows that, across the reported dimensions, our model covers TLS features while keeping negotiation parameters explicit for concrete test-case generation. Instead of fixing client–server choices as assumptions, the model represents the values involved in parameter negotiation, allowing tests to exercise whether implementations generate and validate those values correctly. It also models features not covered by the compared symbolic models, including generic extension handling and post-handshake KeyUpdate. We validated the model with rule-level tests for message-processing rewrite rules and happy-flow interoperability tests against real TLS libraries.

V. Parameterized Behavior Deviations

RFC-conformance testing must generate test cases for off-specification handshake behavior: malformed messages, negotiation in invalid contexts, or non-conforming endpoints that accept input and continue in a state that differs from the RFC-compliant state. A direct way to obtain such traces is to mutate messages or model behavior arbitrarily [10], [21], but arbitrary mutation is insufficient for RFC-conformance testing. It can obscure which RFC requirement was violated, generate inputs irrelevant to TLS conformance, and leave the accepted endpoint behavior unspecified: which validation check was bypassed, and how the model should represent the endpoint after accepting the invalid input. It also lacks a reusable, parameterized way to express recurring RFC violations, so each violation must be encoded as a one-off test.

We therefore introduce a *Behavior Deviation Language*: a named, parameterized description of an off-RFC behavior that is transformed into rewrite rules. The language covers two kinds of deviations: tester-side deviations modify the messages or message flow produced by the tester-controlled endpoint, whereas target-side deviations describe observed implementation effects. Each behavior-deviation instance automatically generates a deviated version of the corresponding Maude rewrite rule defined in Section IV-A.

A. Behavior Deviation Language

A behavior deviation identifies when an off-RFC behavior may occur and which changes it performs, using the form:

```
BehaviorId: id
Parameters: $x_1 : T_1, ..., $x_n : T_n
Conditions: boolean expression over the rule context
Modification: conjunction of modification primitives
```

`BehaviorId` names the deviated behavior. `Parameters` declare typed variables that are later assigned concrete values to create deviation instances, where `T` names a value domain provided by the framework or defined by the user.

`Conditions` can constrain the event kind (`event`), the handled message type (`ruleMsgType`), the matched rule label (`label`), and endpoint-local fields available in the matched context. These predicates can access object attributes, structured fields, and indexed fields such as `keyShareGroup[0]`.

`Modification` describes the behavioral change using primitive operations that edit TLS message fields, suppress validation checks, or update endpoint-local state. There are six primitive modification operations.

- $\mathsf{setM}(f, v)$ replaces the value of message field f with v.
- $\mathsf{addM}(f, v)$ inserts TLS message field f with value v.
- $\mathsf{removeM}(f)$ removes TLS message field f.
- $\mathsf{skip}()$ omits the current message.
- $\mathsf{noCheck}(\lambda)$ disables the RFC-labeled validation check λ
- $\mathsf{setS}(a, v)$ updates state attribute a to v.

Here, f and a range over TLS message-field identifiers and endpoint-state attributes, respectively; λ ranges over RFC requirement checks. Value arguments v must have the declared type of the corresponding field or attribute.

The following example shows a deviation instance `removeKeyShare`. `Conditions` restricts it to resumption `server-hello` send events; and `Modification` removes `key_share` from the constructed message.

```
BehaviorId: removeKeyShare
Conditions: event = send and
  ruleMsgType = server-hello and resumption = true
Modification: remove(key_share)
```

The following example shows another instance, `missingKeyShare`, about accepting an invalid message. It applies to resumption `client-hello` receive events where no key-share group has been selected, disables the `keyShare-exists` check, and sets the selected group to the first supported key-share group.

```
BehaviorId: missingKeyShare
Conditions: event = receive and
  ruleMsgType = client-hello and
  selectedGroup = noGroup and resumption = true
Modification: noCheck(keyShare-exists) and
  setS(selectedGroup, keyShareGroup[0])
```

B. Transforming Deviations into Maude Rules

We make the deviation executable by applying it to matching RFC-compliant rewrite rules from Section IV, producing additional rewrite rules. For each behavior deviation, the transformation selects the RFC-compliant rules whose event kind and message type match `Conditions`, then copies each selected rule. Any remaining object-field predicates in `Conditions` are inserted into the copied rule's `if` condition, so they act as runtime guards. `Modification` edits the copied rule according to its primitives, and `BehaviorId` becomes the generated rule label. For multiple deviations, the transformation can be applied sequentially: rules generated by one deviation become source rules for the next, yielding rules that combine their effects.

For rule instances following the receive rule pattern in Section IV-A, the noCheck and setS primitives transform each instance by changing its condition or result term. $\mathsf{noCheck}(\lambda)$ changes the rule condition to $\texttt{checkRFC}(\Lambda \setminus \{\lambda\}, \texttt{MSG}, ctx)$. $\mathsf{setS}(a, v)$ applies when $a = a_i$ and changes the result-term attribute update from $a_i : v_i'$ to $a_i : v$.

The following generated rule illustrates the transformation of `missingKeyShare`. The deviation name becomes the rule label. The event and message-type conditions select the `ClientHello` receive rule; the remaining conditions become guards requiring a resumption with no selected group. The `noCheck` removes `keyShare-exists` from `checkRFC`. The state-update primitive sets the selected group to `keyShareGroup[0]`. The error rule uses the reduced check set, so the removed check no longer triggers rejection.

```
crl [missingKeyShare]:
   < ci to si : Link | msgQueue : MSG :: MSGS >
   < si : Server | status : sready, keyShareGroup : KGL,
     nonceCtr : N, selectedGroup : noGroup, ... >
=> < ci to si : Link | msgQueue : MSGS >
   < si : Server | status : shello,
     selectedGroup : KGL[0], ... >
if selectedGroup(< si : Server | >) = noGroup
/\ resumption(< si : Server | >) = true
/\ checkRFC({...},MSG,< si : Server | >) .
```

VI. Scenario Properties

Behavior deviations make off-RFC behavior explicit, but test generation also needs to place each deviation in the handshake context where the relevant RFC constraint is checked and to steer the execution toward an outcome such as rejection, target-side acceptance, or completion under an invalid message. We therefore define a scenario property as a finite-trace objective over the labeled transition system from Section IV. A scenario property combines state propositions over TLS configurations with action propositions over rewrite-rule applications, then orders them with sequencing, finite repetition, and non-deterministic choice. The framework Section VII compiles each scenario property into a Maude strategy.

State propositions describe object fields in a TLS configuration. An atomic predicate compares one field of one object. A proposition holds at a trace position when the selected object fields have the expected values.

```
StateProposition [name]:
    boolean expression over endpoint fields
```

The following state propositions marks three states: `resumption` for a resumed start, `errorState` for an endpoint error, and `complete` for handshake completion.

```
StateProposition [resumption]:
    (ci.status = cinit) and (ci.resumption = true)
StateProposition [errorState]:
    (ci.status = error) or (si.status = error)
StateProposition [complete]:
    (ci.status = done) and (si.status = done)
```

Action propositions specify which rule applications may occur at transition positions in a finite trace. They reuse the rule-context predicate syntax of `Conditions` in Section V-A, but use it for trace constraints rather than rule selection.

```
ActionProposition [name]:
    boolean expression over rule context
```

The following action propositions match three transitions: sending the deviated `ServerHello`, accepting it, and taking the `ServerHello` error path.

```
ActionProposition [removeKeyShare]:
  label = removeKeyShareExtension
ActionProposition [acceptInvalidKeyShare]:
  label = acceptMissingKeyShare
ActionProposition [serverHelloError]:
  event = error and ruleMsgType = server-hello
```

A scenario property defines an ordered trace objective as a scenario expression. Each atomic step is a Boolean expression over state and action propositions. Sequencing ; orders required steps, repetition $(\cdot)^*$ allows finite intermediate steps, and choice | allows either branch.

```
ScenarioProperty [name]: scenario expression
```

The following scenario properties define two ordered trace objectives. `rejectKeyShareInResumption` checks the expected rejection path: after resumption and the deviated `ServerHello`, the trace must reach a `ServerHello` error transition and an error state. `losingForwardSecrecy` checks the exploitability path: after the deviated `ServerHello`, the client accepts the message and the handshake completes.

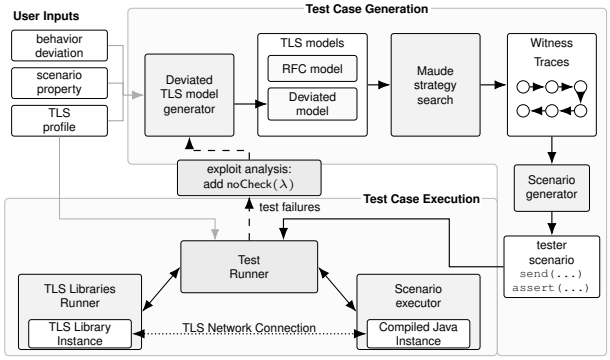

Fig. 4. Framework Architecture.

```
ScenarioProperty [rejectKeyShareInResumption]:
  any* ; resumption ; any* ; removeKeyShare ;
  any* ; serverHelloError; errorState

ScenarioProperty [losingForwardSecrecy]:
  any* ; resumption ; any* ; removeKeyShare ;
  any* ; acceptInvalidKeyShare ; any* ; complete
```

VII. Deviation based Testing Framework

The previous sections define the model-level constructs for deviation-guided testing: an RFC-precise Maude model, behavior deviations, and scenario properties. This section describes the framework that carries these constructs to implementation-level evidence. The framework builds a deviated TLS model, searches it for witness traces, and projects each witness to tester-visible actions that can be executed as black-box tests against an unmodified TLS library. We describe the framework architecture, the generation and execution workflow, the methodology for specification-violation testing and exploitability analysis, and two concrete cases that connect deviations to implementation-level outcomes.

A. Tool Chain

Figure 4 illustrates the workflow that generates test cases from the Maude TLS model, behavior deviations, and scenario properties, and executes them as black-box tests against TLS libraries. The workflow also takes a TLS profile that fixes the protocol version, cryptographic options, certificates, and endpoint roles for the tester and target library.

For test-case generation, the framework instantiates parameterized behavior deviations by enumerating values from their parameter domains: built-in domains for framework-provided types and user-provided domains for user-defined types. It then applies the deviation transformation from Section V to the RFC-compliant rule set from Section IV, producing a model with both RFC-compliant and deviated rules. The scenario property from Section VI is compiled into a Maude strategy, and Maude searches the combined model for witness traces. Because black-box tests control only the tester endpoint, the framework projects witness traces to tester actions: messages to send and assertions over received messages.

For test-case execution, the test runner takes the tester scenario and TLS profile. The scenario executor instantiates the scenario as a TLS-Attacker-based Java test [12], and

the TLS library runner launches TLS libraries as an process configured from the same profile. The two endpoints then interact over a normal TLS network connection. The workflow records the inputs, witness trace, generated scenario, exchanged messages, and observed target behavior. Test failures can trigger exploitability analysis, which augments the model with a target-side validation-bypass deviation and reruns the same generation and execution workflow.

B. Methodology

Using this framework, we first run specification-violation tests against TLS libraries and then, for tests that fail because the target accepts invalid behavior, run exploitability analysis to determine whether the accepted behavior can reach completion or another security-relevant outcome.

For specification-violation testing, the framework provides four parameterized behavior deviations to cover recurring off-specification tester behaviors within the modeled TLS handshake scope, rather than to define an exhaustive taxonomy of TLS bugs. These patterns are motivated by our 2020–2026 CVE analysis of three TLS libraries: among 34 CVEs within the modeled handshake scope, P1–P4 express 30 triggers, with the remaining four handled by case-specific deviations:

- **P1** inserts a disallowed extension component with add.
- **P2** omits a required extension with remove.
- **P3** replaces a modeled field value with setM.
- **P4** omits an expected message with skip.

To generate specification-violation tests from these four deviation patterns, the framework provides the action propositions and scenario property. `deviationPatterns` matches any rule generated from P1–P4, and `alert` matches an error transition. `specViolation` searches for a trace in which a generated deviation is followed by the rejection behavior.

```
ActionProposition [deviationPatterns]:
  label = one of the four behavior-deviation patterns
ActionProposition [alert]:
  event = error
ScenarioProperty [specViolation]:
  any* ; deviationPatterns ; any* ; alert ; errorState
```

If the target accepts the malformed input instead of taking the rejection path, the run is treated as a specification-violation test failure and becomes a candidate for exploitability analysis.

A failed specification-violation test is only an exploitability candidate: the target accepted malformed input, but may still abort before handshake completion. When the triggering tester-side deviation can be associated with the label λ of an RFC validation check, the framework models the observed acceptance as a target-side behavior deviation using noCheck(λ). The continuation scenario asks whether the same malformed tester behavior and modeled target-side acceptance can reach handshake completion, which can be security-relevant state:

```
ActionProposition [triggeringDeviation]:
  label = observed tester-side deviation instance
ActionProposition [acceptInvalidMsg]:
  label = target-side noCheck deviation
ScenarioProperty [analyzeExploitability]:
  any* ; triggeringDeviation ; any* ; acceptInvalidMsg ;
  any* ; complete
```

A witness to `analyzeExploitability` generates a second black-box test. If the target again accepts the malformed input and reaches completion or the specified security-relevant state, the violation is reported as exploitable for that scenario. If a validation-check label alone does not capture the observed target behavior, the user can explicitly provide a more specific target-side deviation and scenario.

C. Case Study: Forward Secrecy Violation

This case revisits the motivating example from Section III, instantiating the two scenario properties from Section VI and executing them against WolfSSL as the target TLS 1.3 client. With `rejectKeyShareInResumption`, the tester acts as the server and sends a resumed `ServerHello` without the required `key_share`. The RFC model expects the client to reject this message, send an alert, and enter `errorState`. The client sent no alert and continued the handshake, so the run was reported as a specification-violation test failure.

The second property, `losingForwardSecrecy`, models this observed acceptance as the target-side behavior and asks whether the execution can reach `complete`. In the second client run, the handshake completed, showing that the invalid acceptance can complete as a resumed session without the requested forward-secret mode.

D. Case Study: Predictable Compromised Key Generation

This case concerns the WolfSSL 5.8.4 TLS 1.3 client behavior associated with CVE [22]. It occurs during the initial TLS 1.3 handshake, not during session resumption. The tester acts as the server and drives the client onto the `HelloRetryRequest` path, where the client retries parameter negotiation with a second `ClientHello`. The tester then sends the subsequent `ServerHello` without the required `key_share`. A conforming client should reject this `ServerHello`; the vulnerable WolfSSL client instead accepts it and continues the handshake. In that continuation, the client derives handshake traffic keys from zero values instead of the server's `key_share`, making the keys predictable to a man-in-the-middle adversary. As a result, a man-in-the-middle adversary can predict the handshake traffic keys and decrypt or modify protected handshake messages.

To generate this case, the framework instantiates the P2 pattern for the `key_share` extension in `ServerHello`. When the black-box test shows that WolfSSL accepts the malformed message, the framework models the target-side behavior, which removes the `key_share` presence check.

```
BehaviorId   : acceptMissingKeyShareExtension
Conditions   : event = receive and
               ruleMsgType = server-hello
Modification : noCheck(key-share-existence)
```

We then apply `analyzeExploitability` to the failed test, combining the original P2 tester-side deviation with the target-side deviation modeling the observed acceptance, and ask whether the client can reach `complete` after accepting the `ServerHello` without `key_share`. In the second WolfSSL run, the handshake completed, so the accepted invalid behavior led to predictable handshake traffic keys.

TABLE II
TEST GENERATION TIME AND COUNTS SUMMARY BY TLS VERSION

TLS Version	Total Time	# Test Cases	Total Time / Test Case
TLS 1.2	2.86h	952	10.8s
TLS 1.3	27.8h	2441	40.9s

VIII. EVALUATION

We evaluate the framework as a specification-violation testing and exploitability-analysis for real TLS implementations. The evaluation answers four research questions: **RQ1**: Can the framework discover previously unknown specification-violation bugs in recent stable TLS libraries? **RQ2**: Can failed specification-violation tests be driven to handshake completion, making the violation exploitable in a concrete execution? **RQ3**: Can the framework reproduce bugs reported by existing TLS testing tools and find additional bugs under the same library and version settings? **RQ4**: Can the framework reproduce known TLS CVEs using deviation-guided scenarios? We ran all experiments on a machine with an Intel Xeon 2.8GHz CPU and 256 GB of memory.

A. New Specification-Violation Bugs and Exploitability

We generate specification-violation tests to identify RFC requirements that recent stable TLS libraries fail to enforce and expose the resulting previously unknown bugs. A specification violation alone does not establish exploitability: the target may accept the malformed input but abort before completion. For accepted violations, we therefore carry the violation into exploitability analysis by continuing the handshake on the same library and checking whether it reaches completion or a security-relevant outcome.

We instantiate the four parameterized behavior-deviation patterns P1–P4 and generate specification-violation tests using the `specViolation` scenario property from Section VII-B. We execute the generated tests against recent stable versions of three TLS libraries: WolfSSL, MbedTLS, and GnuTLS. We count a *test failure* when the target library does not follow the RFC-modeled rejection behavior for the malformed input. For each accepted violation, we generate an exploitability-analysis test using `analyzeExploitability` and execute it.

Table II summarizes the generation cost for the specification-violation test set used in this experiment. The framework generated 952 TLS 1.2 cases and 2,441 TLS 1.3 cases. The TLS 1.3 set is larger and takes longer to generate because the model includes extension features not present in TLS 1.2 and reaches deeper protocol states.

Table III summarizes failed test executions, handshake-completion behavior, and corresponding RFC violations by library and TLS version. For each TLS version, *Test Failure* counts tests in which the target does not follow the RFC-modeled rejection behavior, and *Handshake Completion* counts how many reach handshake completion. *RFC Violation* counts unique violated RFC requirements among the failed tests; the parenthesized number counts how many of those unique violations have at least one execution reaching handshake completion. Overall, 58 TLS 1.2 and 142 TLS 1.3 test failures correspond to 12 and 20 unique RFC violations, of which 8 and 18 have at least one completing execution.

TABLE III
RFC VIOLATIONS AND EXPLOITABILITY RESULTS

Library	TLS 1.2			TLS 1.3		
	Test Failure	Handshake Completion	RFC Violation	Test Failure	Handshake Completion	RFC Violation
WolfSSL 5.8.2	14	8	8 (4)	25	10	10 (10)
MbedTLS 4.0.0	30	30	3 (3)	89	53	5 (4)
GnuTLS 3.8.11	14	14	1 (1)	28	27	5 (4)
Total	**58**	**52**	**12 (8)**	**142**	**90**	**20 (18)**

TABLE IV
RESULTING CONFIRMED BUGS AND CVES

Library	TLS 1.2		TLS 1.3		Total	
	Confirmed	CVEs	Confirmed	CVEs	Confirmed	CVEs
WolfSSL 5.8.2	7	1	8	5	15 / 13	6 / 6
MbedTLS 4.0.0	2	1	4	1	6 / 2	2 / 1
GnuTLS 3.8.11	0	0	4	1	4 / 1	1 / 1
Total	9	2	16	7	**25 / 16**	**9 / 8**

WolfSSL shows how RFC violations connect test failures to handshake completion. In TLS 1.3, 25 test failures map to 10 unique RFC violations, and every violation has at least one completing execution. Thus, the table identifies which unenforced RFC requirements can be carried beyond invalid acceptance, not how many tests fail. MbedTLS shows the complementary case: in TLS 1.2, 30 completing executions arise from 3 unique RFC violations, showing that one unenforced requirement can produce many completion paths. Handshake completion alone does not imply a security-relevant outcome, but it shows that the accepted violation survives later protocol logic. These RFC violations therefore become useful inputs to exploitability analysis and developer triage.

Table IV reports developer-confirmed bugs and assigned CVEs. The TLS 1.2 and TLS 1.3 columns split results by protocol version. In the *Total* columns, the number before the slash counts all confirmed issues in the developer reports, and the number after the slash counts the subset directly attributable to RFC violations found by our framework. Overall, developers confirmed 25 bugs and assigned 9 CVEs across these reports; our framework newly found 16 of those bugs, including 8 CVEs. WolfSSL accounts for the largest share, with 13 of 15 confirmed bugs and all 6 CVEs attributable to our findings. MbedTLS contributes 2 confirmed bugs and 1 CVE, and GnuTLS contributes 1 confirmed bug and 1 CVE.

The smaller number of confirmed bugs and CVEs compared with the RFC-violation counts does not make the remaining RFC violations unimportant. An RFC violation is the requirement-level evidence that a library failed to enforce the specification. For example, duplicate-extension RFC violations in MbedTLS and GnuTLS were either not confirmed as

TABLE V
COMPARISON WITH PRIOR TLS TESTING TOOLS ON RFC VIOLATIONS

Tool	Base. ($\lvert V_T \rvert$)	Ours ($\lvert V_O \rvert$)	$\lvert V_T \setminus V_O \rvert$	$\lvert V_O \setminus V_T \rvert$	$\lvert V_T \cap V_O \rvert$
TLS-Anvil	23	82	7	66	16
TLS-Attacker	7	18	4	15	3
DY-Fuzzer	6	56	3	53	3

V_T denotes the set of RFC violations found by each baseline tool, and V_O denotes the set of RFC violations found by our framework.

bugs or confirmed without CVE assignment, while the same class of violation in WolfSSL contributed to a CVE-assigned denial-of-service case. These results show how RFC-violation detection supports developer triage: violations identify the failed requirements, and handshake-completion results show which accepted violations continue far enough to be examined for security-relevant outcomes. This evidence led to 16 newly found confirmed bugs, including 8 assigned CVEs.

Answer to RQ1. The framework exposed RFC violations in recent stable TLS libraries that led to 16 previously unknown, developer-confirmed bugs.

Answer to RQ2. For accepted RFC violations, exploitability analysis found security-relevant completing executions for 8 CVE-assigned findings.

B. Comparison with Existing Tools on RFC Violations

The comparison with existing TLS testing tools evaluates whether deviation-guided RFC testing reproduces baseline-reported RFC violations and finds additional ones under matched library-version settings. We reuse the specification-violation tests from Section VIII-A, For each comparison target, we execute these tests against the same library version and record which baseline-reported RFC violations are reproduced and which additional RFC violations are observed.

We use the library-version pairs reported by TLS-Anvil [11], TLS-Attacker [12], and DY-Fuzzer [10]: thirteen from TLS-Anvil and five each from TLS-Attacker and DY-Fuzzer. For TLS-Anvil, we derive the baseline RFC-violation counts from both the reported paper results and our reproduced runs. For TLS-Attacker and DY-Fuzzer, we could not reproduce the original experiments, so we derive their baseline RFC-violation counts from the papers.

Table V compares the sets of RFC violations found by prior TLS testing tools with those identified by our framework over the same library-version pairs. Across all three comparisons, $|V_O|$ is larger than $|V_T|$: e.g., 82 versus 23 for TLS-Anvil and 18 versus 7 for TLS-Attacker. The $|V_O \setminus V_T|$ counts show 66, 15, and 53 additional RFC violations identified by our framework under the same settings, while $|V_T \cap V_O|$ shows 16, 3, and 3 overlapping RFC violations. The nonzero $|V_T \setminus V_O|$ counts show 7, 4, and 3 baseline RFC violations not reproduced by our framework in this experiment.

The results indicate that deviation-guided RFC testing finds violations beyond prior tool-specific tests. This is most visible against TLS-Anvil: despite the largest baseline set, our framework identifies many additional RFC violations under the same library-version settings. These additional findings come from modeling RFC requirements in their handshake contexts and instantiating parameterized behavior deviations. In TLS-Anvil, comparable checks are encoded as manual template scenarios.

The non-reproduced violations clarify the scope boundary of our current model. They require behavior outside the RFC-level handshake model, such as byte-level record processing, cryptographic-value construction, legacy protocol features, or unsupported build configurations. We therefore evaluate findings only when their required TLS behavior falls within the current handshake model; out-of-scope cases require additional modeling, not a different testing principle.

Answer to RQ3. Our framework finds 66/15/53 additional RFC violations beyond TLS-Anvil/TLS-Attacker/DY-Fuzzer, while 7/4/3 baseline violations fall outside the current scope.

TABLE VI
REPRODUCTION OF PREVIOUSLY REPORTED CVES

CVE	Target	Exploit Category	Behavior Deviation	Reproduced Tests
[23]	GnuTLS 3.7.9	Memory crash	P2	5
[24]	WolfSSL 5.6.0	Key compromise	P2	1
[25]	WolfSSL 5.5.0	Memory crash	P3	3
[26]	WolfSSL 5.1.0	Client impersonation	P3	5
[27]	WolfSSL 5.1.0	Client impersonation	P4	1
[28]	WolfSSL 5.0.0	Infinite loop	P3	1
[29]	WolfSSL 4.6.0	Server impersonation	P3	10
[30]	WolfSSL 4.4.0	Server impersonation	P4	1
[31]	MbedTLS 4.0.0	Downgrade attack	P3	5

C. Deviation-Guided Reproduction of Known CVEs

The known-CVE reproduction study evaluates whether behavior deviations and scenario properties can express real TLS vulnerability triggers within the modeled handshake scope. For each vulnerable version, the framework builds a deviation-guided scenario from the public CVE or advisory description and checks whether the generated test reproduces the reported trigger and vulnerability outcome.

We include CVEs whose reported triggers fall within the modeled handshake scope. A CVE is reproduced when the generated run exhibits the vulnerable behavior listed in Table VI. For CVEs with multiple plausible malformed inputs, we generate multiple test cases. Table VI summarizes the CVEs, target versions, deviation patterns, and generated tests.

The framework reproduces all nine selected CVEs, using different behavior-deviation forms. In GnuTLS, P2 removes a pre-shared-key field in a resumption context and triggers a null-pointer crash. In the WolfSSL 5.1.0 client-impersonation CVEs, P3 changes the signature algorithm used for certificate verification, while P4 skips the certificate-verification message; both bypass certificate verification. These cases show that the same deviation language can reproduce CVEs ranging from memory crashes to authentication bypasses. Although P1 does not appear in these reproduced CVEs, it was used to find a new WolfSSL CVE in RQ1.

Answer to RQ4. The framework reproduces all nine in-scope CVEs on vulnerable target versions, covering six reported vulnerability classes across 32 generated tests.

IX. Threats to Validity

Internal Validity: The main internal threat is the correctness of the formal model and the workflow that turns it into tests. Because the model is the oracle, an incorrectly encoded RFC requirement can make a compliant target appear to violate the RFC. A fault anywhere in the workflow that transforms the model into executable tests can likewise misclassify a generated test, so a reported violation may not be a real one.

We mitigate this by validating the model and toolchain with rule-level, negative, and interoperability tests (detailed in our artifact [13]), and by deriving every test from a recorded witness trace, so each violation traces back to a specific modeled requirement. These procedures give reproducible evidence, over tested cases, that the model captures intended RFC-level behavior, though not proof of RFC equivalence.

Construct Validity: A first concern is what the evaluation counts as a specification violation. We deduplicate failing tests by the violated RFC requirement, so many inputs exercising one requirement count as a single violation. Developer confirmation and CVE assignment are successively stricter stages reflecting external judgments, not our own. We also separate issues attributable to our reports from all advisory issues.

A second concern is what handshake completion demonstrates. We measure whether a violation can reach a completed handshake, showing that it is not rejected before completion and persists to a working session. We do not treat completion as evidence of security impact: whether a completed violation has a security consequence is case-specific, so we discuss such cases individually rather than count them.

External Validity: The main external threat is scope. The formal model and deviations cover RFC-level TLS 1.2 and 1.3 handshake behavior, but not behavior such as byte-level parsing or cryptographic-oracle semantics. Violations from prior tools or known CVEs that we do not reproduce lie outside this scope; they are not in-scope cases we missed. Modeled requirements and coverage are provided in our artifact [13].

The larger violation counts in Table V do not imply greater effectiveness, since tools target different bug classes while ours targets RFC-level violations. We compare on the same library versions, re-running TLS-Anvil and using reported results for the configuration-sensitive fuzzers (DY-Fuzzer, TLS-Attacker). The comparison thus shows only that we find more RFC-level violations, not greater effectiveness overall.

Finally, because the framework generates black-box tests through the standard TLS interface, our testing approach applies directly to other TLS libraries.

X. Related Work

TLS Implementation Testing: Testing TLS implementations spans fuzzing, workflow construction, combinatorial testing, and differential testing [5]. Protocol fuzzers guided by code coverage or protocol-state feedback mutate message sequences to reach new code or protocol states [32], [33]; without an RFC-level oracle, however, they can miss *silent acceptances*, where an implementation accepts an RFC-invalid message and completes the handshake without visible failure.

Workflow and parameter-based tools build structured TLS interactions: TLS-Attacker [12] supports non-standard workflows, TLS-Anvil [11] tests conformance via RFC-derived templates and combinatorial parameters, and DY-Fuzzer [10] generates tests from Dolev-Yao attacker traces. Differential and RFC-directed approaches compare libraries or generate inputs from selected RFC requirements [34]–[36]. Unlike these, we use an executable formal model as an oracle tied to RFC requirements, and carry accepted violations into deviation-guided exploitability scenarios.

State-Based Analysis and Attack Synthesis: State-learning approaches infer implementation automata and have uncovered state-machine bugs in TLS implementations [6], [8], [37]–[40]. They discover what an implementation *does*, whereas we start from what the RFC *allows*. A related direction synthesizes attacks from formal models or specification-derived state machines [41]–[43], reasoning within the model.

ProInspector [9] tests implementations against a reference state machine with Dolev-Yao-style inputs and uses symbolic provers to check whether nonconforming traces break security properties, again at the model level. Our distinction is a single executable RFC-level oracle supporting both conformance testing and exploitability analysis, with accepted violations carried through to concrete execution on deployed libraries.

TLS Formal Analysis: TLS has been analyzed extensively at the protocol-design level, using formal models to verify handshake and security properties [3], [4], [15], [44], [45]. We use a formal model for a different purpose: not to verify a design, but as an RFC-level oracle to generate conformance tests and assess whether detected violations reach exploitable outcomes on unmodified libraries.

Specification-Guided Testing Beyond TLS: Formal models and specifications have guided implementation testing through model checking, specification-guided test generation, fuzzing, trace validation, and symbolic execution [46]–[51]. Recent LLM-based work also uses RFCs or specifications to audit implementations and guide vulnerability detection or fuzzing [52]–[55]. We share this premise but focus on TLS message validation using executable RFC-level semantics.

XI. Concluding Remarks

Detecting specification violations and establishing their exploitability have usually been separate activities, disconnected from deployed code. We have closed this gap with a deviation-guided testing framework over an executable, RFC-level model of the TLS 1.2 and 1.3 handshake, where behavior deviations and scenario properties express off-specification behavior and exploitable outcomes as explicit test-generation targets. Applied to widely deployed TLS libraries, it has uncovered real, exploitable violations that prior testing misses—8 of them assigned new CVEs—and more RFC-level violations than existing tools, showing that silent-acceptance flaws, invisible to crash-oriented testing, are prevalent and exploitable in production code. Future work includes broadening the model and deviation language, evaluating more implementations, and automating the continuations that still require manual effort.

References

[1] T. Dierks and E. Rescorla, "The transport layer security (tls) protocol version 1.2." RFC 5246, 2008.

[2] E. Rescorla, "The transport layer security (tls) protocol version 1.3." RFC 8446, 2018.

[3] C. Cremers, M. Horvat, J. Hoyland, S. Scott, and T. van der Merwe, "A comprehensive symbolic analysis of tls 1.3," in *Proc. CCS'17*, pp. 1773–1788, ACM, 2017.

[4] K. Bhargavan, V. Cheval, and C. Wood, "A symbolic analysis of privacy for tls 1.3 with encrypted client hello," in *Proc. CCS'22*, pp. 365–379, ACM, 2022.

[5] B. Swierzy, F. Boes, T. Pohl, C. Bungartz, and M. Meier, "SoK: Automated software testing for TLS libraries," in *Proc. ARES'24*, pp. 1–12, ACM, 2024.

[6] J. de Ruiter and E. Poll, "Protocol state fuzzing of TLS implementations," in *Proc. USENIX Security'15*, USENIX Association, 2015.

[7] P. Fiterau-Brostean, B. Jonsson, K. Sagonas, and F. Tåquist, "Automata-based automated detection of state machine bugs in protocol implementations," in *Proc. NDSS'23*, 2023.

[8] P. Fiterau-Brostean, B. Jonsson, K. Sagonas, and F. Tåquist, "SM-BugFinder: An automated framework for testing protocol implementations for state machine bugs," in *Proc. ISSTA'24*, ACM, 2024.

[9] Z. Zhang, L. Jia, and C. Pasareanu, "Proinspector: Uncovering logical bugs in protocol implementations," in *Proc. EuroS&P'24*, IEEE, 2024.

[10] M. Ammann, L. Hirschi, and S. Kremer, "Dy fuzzing: Formal dolev-yao models meet cryptographic protocol fuzz testing," in *Proc. SP'24*, IEEE, 2024.

[11] M. Maehren, P. Nieting, S. Hebrok, R. Merget, J. Somorovsky, and J. Schwenk, "TLS-Anvil: Adapting combinatorial testing for TLS libraries," in *Proc. SEC'22*, USENIX Association, 2022.

[12] J. Somorovsky, "Systematic fuzzing and testing of tls libraries," in *Proc. CCS'16*, pp. 1492–1504, ACM, 2016.

[13] "Supplementary materials." https://maudetls2027-dotcom.github.io/maudetls2027.github.io/, 2026.

[14] D. Dolev and A. C. Yao, "On the security of public key protocols," *IEEE Transactions on Information Theory*, vol. 29, no. 2, pp. 198–208, 1983.

[15] C. Cremers, M. Horvat, S. Scott, and T. van der Merwe, "Automated analysis and verification of TLS 1.3: 0-rtt, resumption and delayed authentication," in *Proc. IEEE S&P'16*, IEEE, 2016.

[16] D. Adrian, K. Bhargavan, Z. Durumeric, P. Gaudry, M. Green, J. A. Halderman, N. Heninger, D. Springall, E. Thomé, L. Valenta, B. VanderSloot, E. Wustrow, S. Zanella-Béguelin, and P. Zimmermann, "Imperfect forward secrecy: How diffie-hellman fails in practice," in *Proc. CCS'15*, ACM, 2015.

[17] National Vulnerability Database, "CVE-2025-11935." https://nvd.nist.gov/vuln/detail/CVE-2025-11935, 2025.

[18] M. Clavel, F. Duran, S. Eker, P. Lincoln, N. Marti-Oliet, J. Meseguer, and C. Talcott, *All About Maude – A High-Performance Logical Framework*, vol. 4350 of *LNCS*. Springer, 2007.

[19] M. Clavel, F. Durán, S. Eker, S. Escobar, P. Lincoln, N. Martı-Oliet, J. Meseguer, R. Rubio, and C. Talcott, "Maude manual (version 3.1)," tech. rep., SRI International, Menlo Park, 2020.

[20] J. Meseguer, "Twenty years of rewriting logic," *Journal of Logic and Algebraic Programming*, vol. 81, no. 7, pp. 721–781, 2012.

[21] F. Dadeau, P.-C. Héam, R. Kheddam, G. Maatoug, and M. Rusinowitch, "Model-based mutation testing from security protocols in HLPSL," *Software Testing, Verification and Reliability*, 2015.

[22] National Vulnerability Database, "CVE-2026-3230." https://nvd.nist.gov/vuln/detail/CVE-2026-3230, 2026.

[23] National Vulnerability Database, "CVE-2025-6395." https://nvd.nist.gov/vuln/detail/CVE-2025-6395, 2025.

[24] National Vulnerability Database, "CVE-2023-3724." https://nvd.nist.gov/vuln/detail/CVE-2023-3724, 2023.

[25] National Vulnerability Database, "CVE-2022-39173." https://nvd.nist.gov/vuln/detail/CVE-2022-39173, 2022.

[26] National Vulnerability Database, "CVE-2022-25638." https://nvd.nist.gov/vuln/detail/CVE-2022-25638, 2022.

[27] National Vulnerability Database, "CVE-2022-25640." https://nvd.nist.gov/vuln/detail/CVE-2022-25640, 2022.

[28] National Vulnerability Database, "CVE-2021-44718." https://nvd.nist.gov/vuln/detail/CVE-2021-44718, 2021.

[29] National Vulnerability Database, "CVE-2021-3336." https://nvd.nist.gov/vuln/detail/CVE-2021-3336, 2021.

[30] National Vulnerability Database, "CVE-2020-24613." https://nvd.nist.gov/vuln/detail/CVE-2020-24613, 2020.

[31] National Vulnerability Database, "CVE-2026-25834." https://nvd.nist.gov/vuln/detail/CVE-2026-25834, 2026.

[32] V.-T. Pham, M. Böhme, A. E. Santosa, A. R. Caciulescu, and A. Roychoudhury, "AFLNet: A greybox fuzzer for network protocols," in *Proc. ICST'20*, IEEE, 2020.

[33] R. Natella, "StateAFL: Greybox fuzzing for stateful network servers," *Empirical Software Engineering*, 2022.

[34] Y. Chen and Z. Su, "Guided differential testing of certificate validation in SSL/TLS implementations," in *Proc. ESEC/FSE'15*, ACM, 2015.

[35] C. Chen, C. Tian, Z. Duan, and L. Zhao, "RFC-directed differential testing of certificate validation in SSL/TLS implementations," in *Proc. ICSE'18*, ACM, 2018.

[36] A. Walz and A. Sikora, "Exploiting dissent: Towards fuzzing-based differential black-box testing of tls implementations," *IEEE Transaction on Dependeable and Secure Computing*, vol. 17, no. 2, pp. 278–291, 2015.

[37] J. de Ruiter, "A tale of the OpenSSL state machine: A large-scale black-box analysis," in *Proc. NordSec'16*, 2016.

[38] P. Fiterau-Brostean, B. Jonsson, R. Merget, J. de Ruiter, K. Sagonas, and J. Somorovsky, "Analysis of DTLS implementations using protocol state fuzzing," in *Proc. USENIX Security'20*, USENIX Association, 2020.

[39] M. Maehren, N. Erinola, R. Merget, J. Schwenk, and J. Somorovsky, "Towards internet-based state learning of tls state machines," in *Proc. SEC'25*, pp. 7097–7116, USENIX Association, 2025.

[40] C. M. Stone, S. L. Thomas, M. Vanhoef, J. Henderson, N. Bailluet, and T. Chothia, "The closer you look, the more you learn: A grey-box approach to protocol state machine learning," in *Proc. CCS'22*, ACM, 2022.

[41] M. von Hippel, C. Vick, S. Tripakis, and C. Nita-Rotaru, "Automated attacker synthesis for distributed protocols," in *Computer Safety, Reliability, and Security - 39th International Conference, SAFECOMP 2020, Lisbon, Portugal, September 16-18, 2020, Proceedings* (A. Casimiro, F. Ortmeier, F. Bitsch, and P. Ferreira, eds.), vol. 12234 of *Lecture Notes in Computer Science*, pp. 133–149, Springer, 2020.

[42] M. L. Pacheco, M. von Hippel, B. Weintraub, D. Goldwasser, and C. Nita-Rotaru, "Automated attack synthesis by extracting finite state machines from protocol specification documents," in *43rd IEEE Symposium on Security and Privacy, SP 2022, San Francisco, CA, USA, May 22-26, 2022*, pp. 51–68, IEEE, 2022.

[43] J. Ginesin, M. von Hippel, E. Defloor, C. Nita-Rotaru, and M. Tüxen, "A formal analysis of SCTP: attack synthesis and patch verification," in *33rd USENIX Security Symposium, USENIX Security 2024, Philadelphia, PA, USA, August 14-16, 2024* (D. Balzarotti and W. Xu, eds.), USENIX Association, 2024.

[44] B. Dowling, M. Fischlin, F. Günther, and D. Stebila, "A cryptographic analysis of the TLS 1.3 handshake protocol candidates," in *Proc. CCS'15*, ACM, 2015.

[45] H. Krawczyk, K. G. Paterson, and H. Wee, "On the security of the TLS protocol: A systematic analysis," in *Proc. CRYPTO'13*, 2013.

[46] D. Wang, W. Dou, Y. Gao, C. Wu, J. Wei, and T. Huang, "Model checking guided testing for distributed systems," in *Proc. EuroSys'23*, ACM, 2023.

[47] Y. Gao, D. Wang, W. Dou, W. Feng, Y. Liang, Y. Feng, and J. Wei, "Model checking guided incremental testing for distributed systems," in *Proc. ISSTA'25*, vol. 2, pp. 279–319, ACM, 2025.

[48] E. M. Gulcan, B. K. Ozkan, R. Majumdar, and Srinidhi, "Model-guided fuzzing of distributed systems," in *Proc. OOPSLA'25*, pp. 274–301, ACM, 2025.

[49] R. Tang, X. Sun, Y. Huang, Y. Wei, L. Ouyang, and X. Ma, "SandTable: Scalable distributed system model checking with specification-level state exploration," in *Proc. EuroSys'24*, ACM, 2024.

[50] H. Cirstea, A. Cailliau, and V. Rahli, "Validating traces of distributed programs against TLA+ specifications," in *Proc. SEFM'24*, 2024.

[51] S. Anand, C. S. Pasareanu, and W. Visser, "SymbexNet: Testing network protocol implementations with symbolic execution and rule-based specifications," *IEEE Transactions on Software Engineering*, 2014.

[52] M. Zheng, C. Wang, X. Liu, J. Guo, S. Feng, and X. Zhang, "RFCAudit: Ai agent for auditing protocol implementations against RFC specifications," in *Proc. ASE'25*, IEEE/ACM, 2025.

[53] H. Zhu, J. Li, C. Gao, J. Qian, Y. Dong, H. Liu, L. Wang, Z. Wang, X. Hu, and G. Li, "Specification-guided vulnerability detection with large language models," 2025.
[54] C. Meng, J. Zhou, F. Wang, and X. Li, "Large language model guided protocol fuzzing," in *Proc. NDSS'24*, 2024.
[55] C. Huang, D. Wang, and Z. Q. Zhou, "Llm-assisted model-based fuzzing of protocol implementations," 2025.

APPENDIX A
TLS CLASS SPECIFICATION

We define a shared `TLS` class for fields common to both client and server endpoints; the `Client` and `Server` classes extend this class with role-specific fields. The `Link` class models one directional network channel between two endpoints. Its `msgQueue` attribute stores the ordered list of TLS messages currently queued on that channel. The following declarations show the endpoint and link classes:

```
class TLS | target : TId,
            --- supported features
            cipherSuites : List{CipherSuite},
            sessionId : Nonce?,
            publicKey : Set{Key},
            privateKey : Set{Key},
            certificates : List{Certificate},
            compressions : List{CompressionMethod},
            keyShareGroup : List{NamedGroup},
            extensions : Extension,

            pskInfo : List{PSKInfo},
            --- selected session parameters
            selectedVersion : ProtocolVersion?,
            selectedCipherSuite : CipherSuite?,
            selectedCompression : CompressionMethod?,
            selectedPSK : Nonce?,
            selectedExtensions : Extension,
            selectedGroup : NamedGroup?,
            serverRandom : Nonce?,
            clientRandom : Nonce?,
            sessionReadKey : Key?,
            sessionWriteKey : Key?,
            peerCertificate : List{Certificate},

            --- TLS 1.2-specific features
            sessionState : SessionState,
            sessionResumption : Bool,
            failedSessionId : Nonce?,
            secureRenegotiation : Bool,
            masterSecret : Nonce?,
            keyExchangeGroup : NamedGroup?,
            keyExchangeValue : Nonce?,
            certificateType : List{CertificateType},
            certificateAlgo : List{SignatureAlgorithm},
            certificateAuth : List{TId},
            pendingReadKey : Key?,
            pendingWriteKey : Key?,
            prevClientVerifyData : Nonce?,
            prevServerVerifyData : Nonce?,

            --- TLS 1.3-specific features
            earlySecret : Nonce?,
            sharedSecret : Nonce?,
            handshakeSecret : Nonce?,
            applicationSecret : Nonce?,
            certificateRequestContext : Nonce?,
            keyUpdateReq : Bool,
            keyUpdateReqType : Bool,
            keyUpdateWait : Bool,
            pskDHEMode : List{PreSharedKeyExchangeMode},

            --- runtime metadata
            transcript : MsgContent,
            reconnect : Bool,
            reconnectInProgress : Bool,
            nonceCtr : Nat,
            errorLog : AlertDescription? .

class Client | status : ClientState,
               certificateRequested : Bool,
               newSessionTicketWait : Bool,
               earlyDataReq : Bool,
               postClientAuthReq : Bool,
               clientRenegotiation : Bool .

class Server | status : ServerState,
               clientCertificateReq : Bool,
               newSessionTicketReq : Bool,
               earlyDataWait : Bool,
               postClientAuthRequested : Bool,
               firstClientHello : MsgContent .

subclass Client Server < TLS .

class Link | msgQueue : List{TLSMsg} .
```

APPENDIX B
LABELED TRANSITION SYSTEM

We define the TLS transition system as $M_{\mathsf{TLS}} = (S, A, \rightarrow, s_0)$, where S is the set of well-formed TLS object configurations, A is the set of rewrite-rule applications, $\rightarrow \subseteq S \times A \times S$ is the transition relation, and $s_0 \in S$ is the initial configuration. Each state contains the client and server endpoint objects and the two directional link objects.

An action specifies the context of a rewrite-rule application:

$$a = (e, \ell, m, objs) \in A.$$

Here $e \in \{\mathsf{send}, \mathsf{receive}, \mathsf{error}\}$ is the event kind. The component ℓ is the Maude rewrite-rule label, m is the TLS message type handled by the rule, and $objs$ records the Maude object terms matched by the rule's left-hand side.

The transition relation $s \xrightarrow{a} s'$ holds when a conditional Maude rewrite rule with rule label ℓ, event kind e, and handled message type m matches the objects $objs$ in configuration s, its condition is satisfied, and applying the rule rewrites s to s'. The action a is not an additional semantic step; it is the recorded context of the rewrite step.

For example, applying the `sendClientHello` rule to the client object obj_{c} and the client-to-server link object obj_{c2s} is associated with

$$(\mathsf{send}, \mathsf{sendClientHello}, \mathsf{ClientHello}, \{obj_{\mathsf{c}}, obj_{\mathsf{c2s}}\}).$$

The successor state contains the updated client object and the constructed `ClientHello` appended to the client-to-server queue. Applying the corresponding `receiveClientHello` rule to obj_{c2s} and the server object obj_{s} is associated with

$$(\mathsf{receive}, \mathsf{receiveClientHello}, \mathsf{ClientHello}, \{obj_{\mathsf{c2s}}, obj_{\mathsf{s}}\}).$$

The successor state contains the consumed queue entry and the updated server object.

A finite TLS trace is an alternating sequence of states and actions:

$$\tau = s_0 \xrightarrow{a_0} s_1 \xrightarrow{a_1} \cdots \xrightarrow{a_{k-1}} s_k.$$

APPENDIX C
REUSABLE BEHAVIOR-DEVIATION PATTERNS

RFC-conformance testing spans many TLS message types, extensions, fields, and protocol contexts, but the framework does not try to enumerate every malformed input manually.

Instead, for specification-violation testing, it provides four parameterized tester-side behavior-deviation patterns that capture recurring off-specification message-production behaviors within the modeled TLS handshake scope. These patterns form a reusable generation library, not an exhaustive taxonomy of TLS bugs: each pattern specifies one class of malformed tester behavior, and the framework instantiates that class across concrete TLS contexts by enumerating typed parameter domains, using built-in domains for framework-provided types and user-provided domains for user-defined types.

- **P1:** `addUnexpectedExtension` inserts an extension component that is disallowed in the current message or protocol context.
- **P2:** `removeNecessaryExtension` omits an extension required by the current message or protocol context.
- **P3:** `changeMessageFieldValues` replaces a TLS message-field value while preserving the structure.
- **P4:** `skipExpectedMessage` omits a message expected in the current modeled handshake path.

For example, P1 captures extension-validity violations such as duplicate extensions, forbidden extensions in a message, and unsolicited extension responses. It also covers extensions that are valid in one handshake message but invalid in another. The pattern selects send rules for a chosen TLS message type and inserts an extension field-value component into the constructed message. The expression `#getType( $extField)` returns the value type associated with the selected extension field, so each instantiated `$extValue` is type-consistent with `$extField`. Example parameter instances include adding a `signature_algorithms` extension to a `ServerHello` or adding a `supported_groups` extension to a `CertificateRequest`.

```
BehaviorId   : addUnexpectedExtension
Parameters   : $extField : ExtensionField,
  $extValue : #getType($extField), $msgType : TLSMsgType
Conditions   : event = send and ruleMsgType = $msgType
Modification : add($extField, $extValue)
```

P2 captures missing-extension violations in contexts where the RFC-compliant peer expects mandatory extension information. These cases differ from P1 because the malformed message is invalid by omission rather than by the presence of an unexpected component. The pattern selects send rules for a chosen TLS message type and removes the selected extension field from the constructed message. Example parameter instances include removing `key_share` from a `ClientHello` in a context where that extension is required or removing `signature_algorithms` from a `CertificateRequest`.

```
BehaviorId   : removeNecessaryExtension
Parameters   : $extField : ExtensionField,
  $msgType : TLSMsgType
Conditions   : event = send and ruleMsgType = $msgType
Modification : remove($extField)
```

P3 captures value-validity violations in modeled TLS message fields while preserving the corresponding message structure. The pattern selects send rules for a chosen TLS message type and replaces a selected message field with a type-consistent replacement value. This allows one specification to cover deviations in negotiated values, authentication-related fields, and other modeled TLS fields. The deviation remains at the structured-message level: it changes the field value represented in the Maude message term, not the concrete byte encoding. Example parameter instances include setting `legacy_version` in a `ClientHello` to `TLS11` or changing the `handshakeType` of a `CertificateRequest` to `ServerHello`.

```
BehaviorId   : changeMessageFieldValues
Parameters   : $msgField : MessageField,
  $msgValue : #getType($msgField), $msgType : TLSMsgType
Conditions   : event = send and ruleMsgType = $msgType
Modification : setM($msgField, $msgValue)
```

P4 captures missing-message violations in contexts where the current modeled handshake path requires a particular message next. Once a TLS handshake path is determined, the peer has an expected message order; if the expected message does not appear, the receiver observes an RFC-deviating message flow. Unlike P1–P3, which modify the contents or structure of a message, P4 selects send rules for a chosen TLS message type and omits the corresponding message.

More complex ordering scenarios, such as replaying a previous message, inserting an additional message, or sending a later message in place of the omitted one, can be constructed by combining multiple deviations and scenario properties. They are therefore not represented as a single generic reordering primitive. An example parameter instance omits a server `Certificate` message.

```
BehaviorId   : skipExpectedMessage
Parameters   : $msgType : TLSMsgType
Conditions   : event = send and ruleMsgType = $msgType
Modification : skip()
```

APPENDIX D
RFC 5246 AND RFC 8446 REQUIREMENT COVERAGE

We model RFC requirements at the level of individual MUST and MUST NOT obligations because the Maude model is used as an executable oracle for RFC-conformance test generation, not merely as a high-level protocol sketch. A generated test needs a concrete reason why a message is valid or invalid in a particular handshake context, and the framework must know what a compliant endpoint should do next. Encoding RFC MUST and MUST NOT requirements as explicit send-side construction constraints, receive-side checks, error transitions, or state updates lets the framework derive targeted malformed variants, predict the expected rejection behavior, and relate an implementation's unexpected acceptance back to the violated RFC requirement.

The coverage tables below make this oracle boundary inspectable at the RFC-section level. In each table, rows organize RFC 5246 or RFC 8446 MUST and MUST NOT requirements by specification section and protocol function. The **Reqs.** column reports the number of requirements in the corresponding RFC section group, the **Model** column reports how many of those requirements are explicitly encoded in the Maude model, and the **Cov.** column reports the coverage ratio.

TABLE VII
MODELED COVERAGE OF RFC 8446 REQUIREMENTS BY GROUP

Group	RFC section basis	Reqs.	Model	Cov.
Key Exchange	Sections 2, 4.1	50	42	84.0%
Extensions	Section 4.2	96	71	74.0%
Server Parameters	Section 4.3	8	8	100.0%
Authentication Messages	Section 4.4	41	34	82.9%
Early Data / Post-Handshake	Sections 4.5–4.6	23	16	69.6%
Record / Alert Protocol	Sections 5–6	50	28	56.0%
Crypto / 0-RTT / Compliance	Sections 7–9	30	8	26.7%
Others	Section 4, Appendices B–E	29	22	75.9%
Total		**327**	**229**	**70.0%**

TABLE VIII
MODELED COVERAGE OF RFC 5246 REQUIREMENTS BY GROUP

Group	RFC section basis	Reqs.	Model	Cov.
Record Layer	Section 6	21	6	28.6%
Alert / ChangeCipherSpec	Sections 7.1–7.2	12	11	91.7%
Hello / Negotiation	Sections 7.3–7.4.1	24	19	79.2%
Server Auth / Key Exchange	Sections 7.4.2–7.4.5	21	11	52.4%
Client Auth / Key Exchange	Sections 7.4.6–7.4.8	24	18	75.0%
Finished	Section 7.4.9	4	4	100.0%
Compatibility / Cipher Suites	Section 9, Appendix A.5/E	18	5	27.8%
Others	Sections 1.2, 4.7, 5	5	2	40.0%
Total		**129**	**76**	**58.9%**

A requirement contributes to the **Model** column only when the executable Maude model encodes that requirement as logic for constructing a TLS message, validating a received message, updating handshake state, or rejecting invalid input.

Table VIII shows that the modeled RFC 5246 requirements are concentrated in the parts of TLS 1.2 that determine endpoint-visible handshake behavior: hello negotiation, alert and ChangeCipherSpec handling, certificate and key-exchange messages, client authentication, and Finished processing. These are the requirements that the model must encode to construct valid handshake traces, generate malformed variants, and decide whether a peer should accept or reject a message. Coverage is lower in groups such as Record Layer, Server Auth / Key Exchange, and Compatibility / Cipher Suites because many requirements there specify fragmentation, compression, MAC and encryption processing, cipher-suite definitions, deployment compatibility constraints, or cryptographic validation details rather than the handshake-level oracle used by our generated tests.

Table VII shows that the modeled RFC 8446 requirements are concentrated in the parts of TLS 1.3 that determine endpoint-visible handshake behavior: key exchange, extension negotiation, server parameters, authentication messages, early data and post-handshake messages, alert behavior, and state updates. These are the requirements that the model must encode to construct valid handshake traces, generate malformed variants, and decide whether a peer should accept or reject a message. Coverage is lower in groups such as Record / Alert Protocol and Crypto / 0-RTT / Compliance because many requirements there specify byte-level encoding, cryptographic computations, anti-replay or deployment obligations, rather than the handshake-level oracle used by our generated tests.

APPENDIX E
SEMANTICS OF STATE AND ACTION PROPOSITIONS

Scenario properties constrain finite traces of the labeled transition system in Section B. They use two kinds of predicates. State propositions are evaluated over TLS object configurations, and action propositions are evaluated over the recorded context of rewrite-rule applications. This separation lets a scenario refer both to protocol states, such as an endpoint reaching `error` or `done`, and to transition events, such as applying a particular deviated rule or taking an error transition.

Let $s \in S$ be a TLS object configuration. Since configurations are multisets of Maude objects, we use $\{\langle o : C \mid x : v, \cdots \rangle, \ldots\}$ to denote a configuration that contains an object with identifier o, class C, and attribute x with value v; the omitted attributes and the other objects in the configuration are irrelevant to the proposition. The satisfaction relation for state propositions is:

$$\begin{aligned} s \models o.x = v &\iff s = \{\langle o : C \mid x : v, \cdots \rangle, \ldots\}, \\ s \models \texttt{not}\ \delta &\iff \neg(s \models \delta), \\ s \models \delta_1\ \texttt{and}\ \delta_2 &\iff (s \models \delta_1) \wedge (s \models \delta_2), \\ s \models \delta_1\ \texttt{or}\ \delta_2 &\iff (s \models \delta_1) \vee (s \models \delta_2). \end{aligned}$$

For example, `ci.status = done` holds in a state when the object `ci` has attribute `status` with value `done`. State propositions are evaluated at state positions in a trace, including the final state.

Action propositions are evaluated over the action tuple $a = (e, \ell, m, \mathit{objs})$ from Section B. They inspect either the tuple metadata, such as `event`, `label`, and `ruleMsgType`, or the attributes of the object terms in objs. The satisfaction relation is:

$$\begin{aligned} a \models \texttt{event} = e' &\iff e = e', \\ a \models \texttt{label} = \ell' &\iff \ell = \ell', \\ a \models \texttt{ruleMsgType} = m' &\iff m = m', \\ a \models o.x = v &\iff \mathit{objs} = \{\langle o : C \mid x : v, \cdots \rangle, \ldots\}, \\ a \models \texttt{not}\ \psi &\iff \neg(a \models \psi), \\ a \models \psi_1\ \texttt{and}\ \psi_2 &\iff (a \models \psi_1) \wedge (a \models \psi_2), \\ a \models \psi_1\ \texttt{or}\ \psi_2 &\iff (a \models \psi_1) \vee (a \models \psi_2). \end{aligned}$$

For example, `event = error` selects rejection transitions, `label = removeKeyShareExtension` selects applications of that generated rewrite rule, and `ruleMsgType = server-hello` selects rule applications whose handled TLS message type is `server-hello`. A field predicate such as $o.x = v$ is evaluated over the matched object terms in objs, not over the successor state produced by the rewrite step.

Unlike state propositions, action propositions are evaluated only at transition positions. If a finite trace has k transitions, action propositions can be evaluated only at indices $0, \ldots, k-1$; the final trace position is a state and has no outgoing action.

APPENDIX F
SEMANTICS OF SCENARIO PROPERTY

We define scenario property semantics over the finite TLS traces introduced in Section B. For a trace τ with n transitions, positions range from 0 to n; s_i denotes the state at position i, and a_i denotes the action from s_i to s_{i+1}. Scenario satisfaction is written $\tau, i \models \varphi$, meaning that scenario expression φ is satisfied from trace position i. State-only conditions are evaluated over s_i. Action-only conditions are evaluated over a_i and therefore require $i < n$. Mixed state/action conditions require both parts to hold at the same transition position.

For atomic expressions:

$$\begin{aligned}
\tau, i \models \mathsf{true} &\iff 0 \le i \le n, \\
\tau, i \models \mathsf{false} &\iff \mathsf{false}, \\
\tau, i \models \delta &\iff 0 \le i \le n \land s_i \models \delta, \\
\tau, i \models \psi &\iff 0 \le i < n \land a_i \models \psi, \\
\tau, i \models \delta \text{ and } \psi &\iff 0 \le i < n \land s_i \models \delta \land a_i \models \psi, \\
\tau, i \models \mathsf{any} &\iff 0 \le i < n.
\end{aligned}$$

Here, δ ranges over state propositions and ψ ranges over action propositions, whose meanings are defined in Section E.

For composition:

$$\begin{aligned}
\tau, i \models \varphi_1; \varphi_2 &\iff i < n \land \tau, i \models \varphi_1 \land \tau, i+1 \models \varphi_2, \\
\tau, i \models \varphi^* &\iff \exists k.\ i \le k \le n \land \forall j.\ i \le j < k \\
&\qquad \Rightarrow \tau, j \models \varphi, \\
\tau, i \models (\varphi_1 \mid \varphi_2) &\iff \tau, i \models \varphi_1 \lor \tau, i \models \varphi_2.
\end{aligned}$$

Because traces are finite, repetition in φ^* is always finite.

A trace satisfies a scenario property if the property holds at the initial trace position:

$$\tau \models \varphi \quad \iff \quad \tau, 0 \models \varphi.$$

This semantics treats scenario properties as existential finite-trace objectives: the analysis searches for some finite trace τ such that $\tau \models \varphi$.

APPENDIX G
MANUAL FOLLOW-UP ANALYSIS OF MBEDTLS

The third case shows the manual path for target-specific state progression. The target is an MbedTLS 4.0.0 server configured for TLS 1.3 with client authentication, reconnection, and ticket resumption. The security-relevant outcome is a client-impersonation scenario: after a TLS 1.3 ticket-resumption flow, the server proceeds into a TLS 1.2 resumption-like state whose session secret is modeled as zero.

The initial RFC-conformance test is still generated automatically from the reusable library. The effective tester-side deviation removes a required `supported_versions`-related extension from a `ClientHello` in the resumption path:

```
BehaviorId   : removeNecessaryExtension
Conditions   : event = send and ruleMsgType = client-hello
Modification : remove(supported_versions)
```

RFC 8446 requires that, when a client sends a second `ClientHello` in response to a `HelloRetryRequest`, its `supported_versions` extension be identical to the one in the first `ClientHello`. Since this deviation removes `supported_versions` from the second `ClientHello`, a conforming server should reject it. In concrete execution, the MbedTLS server does not immediately reject it; instead, it sends subsequent handshake messages on a path that resembles TLS 1.2 resumption. The framework classifies the first execution as unexpected acceptance.

After this unexpected acceptance, the framework first applies the same violation-guided follow-up as in Section VII-D. It maps the triggering deviation to the modeled mandatory-extension validation check, adds the corresponding noCheck behavior, and searches for a follow-up trace. However, this automated attempt does not reach the exploit condition. The observed execution therefore requires target-specific behavior beyond removing the RFC-labeled check used in the automated path.

The user then inspects the concrete execution and identifies the missing target-side behavior: after accepting the invalid resumption `ClientHello`, the MbedTLS proceeds as if it had entered a TLS 1.2 resumption mode with a zero-valued session secret. The user expresses this observed behavior as behavior deviation specifications. The first specification below captures the target-side state progression, using setS to encode the protocol-version, handshake-state, and session-state updates needed to model this complex target-side deviation.

```
BehaviorId   : mbedTLSWrongResumptionState
Conditions   : event = receive and
               and ruleMsgType = client-hello
               and reconnectInProgress = true
Modification : setS(server.protocolVersion, TLS-12)
               and setS(server.handshakeState, V2S-READY)
               and setS(server.sessionState, zeroSecret)
```

The second deviation adjusts the controlled tester state so that the generated trace can continue consistently with the target's wrong state progression. It does not modify the target; it describes how the tester should continue the conversation after the target has entered the abnormal resumption path.

```
BehaviorId   : tryTLS12Resumption
Conditions   : event = receive and
               and ruleMsgType = server-hello
               and reconnectInProgress = true
Modification : setS(client.protocolVersion, TLS-12)
               and setS(client.handshakeState, V2C-INIT)
               and setS(client.sessionState, zeroSecret)
```

The custom scenario property requires the initial RFC-violating input, the tester-side resumption adjustment, the target-side wrong-state behavior, and a completed resumption handshake.

```
ScenarioProperty [manualExploitAnalysis]:
    any* ; (label = removeNecessaryExtension) ;
    any* ; (label = mbedTLSWrongResumptionState) ;
    any* ; (label = tryTLS12Resumption) ;
    any* ; (server.serverState = handshake-success
            and server.selectedVersion = TLS-12)
```

With these manually described deviations and the custom scenario property, the framework generates a new trace and

executes the generated concrete test. The MbedTLS server completes the resumed handshake. In this state, the server carries forward the client-authenticated session identity with a zero-valued resumed session secret, thereby accepting the peer as the previously authenticated client. This is the client-impersonation condition expressed by the manual scenario.

This case demonstrates the role of manual behavior specification in complex exploit analysis. The initial RFC violation is detected by the same library-driven path used in the previous cases, but the exploit requires target-specific state information. Once the user expresses that behavior as behavior deviations and a scenario property, the framework uses the same trace-generation and concrete-execution workflow to exploit target library.

APPENDIX H
DETAILED RQ2 RESULTS

APPENDIX I
DETAILED RQ3 EXISTING-TOOL COMPARISON

TABLE IX
RQ2 EXPLOITABILITY MEASUREMENTS BY BUG REFERENCE

Target	Behavior Deviation	Bug Reference	Test Fails	Exploit Tests
MbedTLS(4.0.0)	P2	[?]	1	1
WolfSSL(5.8.4)	P2	[?]	1	1
GnuTLS(3.8.11)	P3	[?]	5	-
WolfSSL(5.8.2)	P3	[?]	2	2
WolfSSL(5.8.2)	P3	[?]	8	8
WolfSSL(5.8.2)	P2	[?]	1	1
WolfSSL(5.8.2)	P3	[?]	2	2
WolfSSL(5.8.2)	P1	[?]	1	1
WolfSSL(5.8.2)	P5	[?]	1	0
WolfSSL(5.8.4)	P5	[?]	1	0
WolfSSL(5.8.4)	P5	[?]	1	0
WolfSSL(5.8.4)	P5	[?]	1	0
WolfSSL(5.8.4)	P4	[?]	1	0
WolfSSL(5.8.4)	P1	[?]	2	2
WolfSSL(5.8.4)	P1	[?]	8	8
MbedTLS(4.0.0)	P1	[?]	30	30

Each row corresponds to one newly discovered bug row from Table XII. Failures are failed tests from **??**; exploit tests measure additional continuation tests.

TABLE X
RQ2 CVE METADATA FOR NEWLY DISCOVERED BUGS

Target	Behavior Deviation	Bug Reference	CVE	CVSS	Vulnerability Category
MbedTLS(4.0.0)	P2	[?]	CVE-2026-34873 [?]	Critical (9.1)	Client impersonation
WolfSSL(5.8.4)	P2	[?]	CVE-2026-3230 [22]	Low (2.7)	Key compromise
GnuTLS(3.8.11)	P3	[?]	CVE-2026-1584 [?]	High (7.5)	Memory crash
WolfSSL(5.8.2)	P3	[?]	CVE-2025-12889 [?]	Medium (5.4)	Downgrade attack
WolfSSL(5.8.2)	P3	[?]	CVE-2025-11936 [?]	Medium (5.3)	Denial of service
WolfSSL(5.8.2)	P2	[?]	CVE-2025-11935 [17]	High (7.5)	Non-perfect forward secrecy
WolfSSL(5.8.2)	P3	[?]	CVE-2025-11934 [?]	Low (2.7)	Downgrade attack
WolfSSL(5.8.2)	P1	[?]	CVE-2025-11933 [?]	Medium (6.5)	Denial of service
WolfSSL(5.8.2)	P3	[?]	-	-	-
WolfSSL(5.8.4)	P5	[?]	-	-	-
WolfSSL(5.8.4)	P5	[?]	-	-	-
WolfSSL(5.8.4)	P5	[?]	-	-	-
WolfSSL(5.8.4)	P4	[?]	-	-	-
WolfSSL(5.8.4)	P1	[?]	-	-	-
WolfSSL(5.8.4)	P1	[?]	-	-	-
MbedTLS(4.0.0)	P1	[?]	-	-	-

CVSS reports public CVSS v3.1 base severity and score in the cited CVE records. Bugs without a linked CVE use "-" for the CVE, CVSS, and vulnerability-category fields.

TABLE XI
REPRODUCTION AND ADDITIONAL BUG DETECTION UNDER EXISTING-TOOL LIBRARY/VERSION SETTINGS

Tool	Target	Repro-duced	Addl. bugs	Test Fails
DY-Fuzzer	OpenSSL 1.1.1j	Y(1)	6	23
	WolfSSL 5.1.0	Y(1)	16	163
	WolfSSL 5.3.0	N(1)	11	92
	WolfSSL 5.4.0	N(1)	9	100
	WolfSSL 5.5.0	P(1/2)	13	121
TLS-Anvil	BearSSL 0.6	N(1)	1	8
	BoringSSL 3945	–	13	31
	Botan 2.17.3	–	2	32
	GnuTLS 3.7.0	Y(1)	6	11
	LibreSSL 3.2.3	P(1/2)	1	15
	MatrixSSL 4.3.0	P(7/11)	13	159
	MbedTLS 2.25.0	N(1)	2	17
	NSS 3.6	–	4	35
	OpenSSL 1.1.1i	–	6	23
	Rustls 0.19.0	Y(1)	2	17
	s2n 0.10.24	Y(1)	6	12
	tlslite-ng 0.8.0-a39	–	2	4
	WolfSSL 4.5.0	Y(5)	13	136
TLS-Attacker	Botan 1.11.21	N(1)	1	26
	OpenSSL 1.1.0-pre1	Y(1)	5	38
	MatrixSSL 3.8.1	N(2)	N/A	N/A
	GnuTLS 3.4.9	Y(1)	3	52
	Botan 1.11.28	Y(1)	1	26
	Botan 1.11.30	N(1)	5	63

Reproduced: Y=yes, N=no, P=partial(reproduced/reported), –=no prior reported bug.
Addl. bugs count unique RFC violations within each library/version setting.

APPENDIX J
RFC REQUIREMENT LABELS

This appendix lists the RFC requirements [1], [2] encoded as label-indexed predicates used by the receive rules in Section IV. Each table row is assigned a globally unique numeric label identifier. The label is a stable handle for the modeled predicate; the RFC reference, message type, and description provide the protocol context in which that predicate is checked.

Receive rules call `checkRFC` with a requirement label, the received message, and the receiver object matched by the rule. The label selects the predicate to evaluate, while the receiver object supplies the endpoint context needed by state-dependent requirements, such as supported parameters or already negotiated values. For a message field $f_1(m_1)$, a representative predicate has the following form:

$$\begin{aligned} &\texttt{eq checkRFC}(l_{f_1},\ f_1(m_1)\cdots f_n(m_n),\\ &\quad \langle o_{dst} : \mathit{NodeType} \mid\ a_1 : v_1, \ldots a_m : v_m\ \rangle)\\ &= \texttt{checkMessage}(m_1, \mathit{opr}(v_1, \ldots v_m)). \end{aligned}$$

The first example below checks a `ClientHello` cipher-suite field against the server's supported cipher suites, whereas the second checks a message-structural requirement independent of endpoint attributes:

```
eq checkRFC(cipher-supported, cipher(CSL) MSGS,
     < si : Server | cipher : CSL' >)
 = CSL == CSL' .
eq checkRFC(key-share-exists, MSGS, < si : Server | >)
 = isKeyShareIncluded(MSGS) .
```

TABLE XII: RFC requirement labels encoded by the Maude TLS model.

Message Type	Label ID	RFC	Section	Description
ClientHello	label1	8446	4.2.8	KeyShareEntry values MUST correspond to groups offered in the "supported_groups" extension and MUST appear in the same order.
ClientHello	label2	8446	4.1.2	TLS 1.3 forbids renegotiation; reject TLS 1.2 renegotiation signaling on TLS 1.3 ClientHello paths.
ClientHello	label3	8446	4.2	There MUST NOT be more than one extension of the same type in a given extension block.
ClientHello	label4	8446	4.1.1	If no supported client/server parameters overlap, the server MUST abort with handshake_failure or insufficient_security.
ClientHello	label5	8446	4.1.1 and 4.2.7	The server must be able to negotiate supported parameters, including a mutually supported named group when groups are offered.
ClientHello	label6	8446	4.1.2	The TLS 1.3 ClientHello compression vector MUST contain exactly one zero byte, the null compression method.
ClientHello	label7	8446	9.2	A TLS 1.3 ClientHello with supported_versions 0x0304 MUST meet mandatory extension requirements.
ClientHello	label8	8446	4.2.3	Deprecated signature algorithms MUST NOT be offered or negotiated; MD5, SHA-224, and DSA MUST NOT be used.
ClientHello	label9	8446	4.1.2 and 4.2.1	TLS 1.3 ClientHello uses legacy_version 0x0303 and identifies TLS 1.3 via supported_versions 0x0304.
ClientHello	label10	8446	4.2.8	Clients MUST NOT offer multiple KeyShareEntry values for the same group.
ClientHello	label11	8446	4.2.3	If the server authenticates via certificate and the client omitted signature_algorithms, the server MUST abort.
ClientHello	label12	8446	B.4	TLS 1.3 cipher suites cannot be used for TLS 1.2, and TLS 1.2 or lower cipher suites cannot be used with TLS 1.3.
ClientHello	label13	8446	4.2.9	A client offering pre_shared_key MUST also provide psk_key_exchange_modes.
ClientHello	label14	8446	4.2.10	A client using early data MUST supply both pre_shared_key and early_data extensions.
ClientHello	label15	8446	4.2.9	In psk_dhe_ke, client and server MUST supply key_share values; in psk_ke, the server MUST NOT use key_share.
ClientHello	label16	8446	4.1.2 and 6	ClientHello vector bounds must parse correctly; syntax length failures terminate with decode_error.
ClientHello	label17	8446	4	The expected handshake message is ClientHello; unexpected handshake order or type aborts with unexpected_message.
ClientHello	label18	8446	5	ClientHello is processed as a handshake record; unexpected record content types terminate with unexpected_message.
HelloRetryRequest	label19	8446	4.1.4	HelloRetryRequest MUST only contain extensions valid for HelloRetryRequest.
HelloRetryRequest	label20	8446	4.1.4	HelloRetryRequest MUST contain supported_versions.
HelloRetryRequest	label21	8446	4.1.4	The selected group must cause a changed ClientHello.
HelloRetryRequest	label22	8446	4.1.4	The selected group must be from the original supported_groups.
HelloRetryRequest	label23	8446	4.1.4	A client MUST abort on a second HelloRetryRequest in the same connection.
HelloRetryRequest	label24	8446	4.1.4	HelloRetryRequest legacy_version MUST be TLS 1.2.
HelloRetryRequest	label25	8446	4.1.4	HelloRetryRequest cipher suite MUST be one offered by the client.
HelloRetryRequest	label26	8446	4.1.4	HelloRetryRequest compression method MUST be null.
HelloRetryRequest	label27	8446	4.1.4	HelloRetryRequest legacy_session_id_echo MUST match ClientHello.
HelloRetryRequest	label28	8446	4.2	There MUST NOT be duplicate extensions.
HelloRetryRequest	label29	8446	4.2.1	HelloRetryRequest selected_version MUST have been offered by the client.
HelloRetryRequest	label30	8446	4 and 6	HelloRetryRequest syntax lengths must parse correctly.
HelloRetryRequest	label31	8446	4	The expected handshake message is HelloRetryRequest.

Message Type	Label ID	RFC	Section	Description
HelloRetryRequest	label32	8446	5	HelloRetryRequest is carried in a handshake record.
ServerHello	label33	8446	4.1.3 and 4.2.1	ServerHello legacy_version and supported_versions must indicate TLS 1.3 correctly.
ServerHello	label34	8446	4.2.1	ServerHello selected_version must be one offered by the client.
ServerHello	label35	8446	4.2	There MUST NOT be duplicate extensions.
ServerHello	label36	8446	4.1.3	ServerHello cipher suite MUST be one offered by the client.
ServerHello	label37	8446	4.2	ServerHello MUST NOT contain extensions forbidden for this message.
ServerHello	label38	8446	4.2	ServerHello extension responses require corresponding client requests.
ServerHello	label39	8446	4.2.8	ServerHello key_share MUST match a client offered group.
ServerHello	label40	8446	4.2.8	ServerHello key_share group MUST appear in client supported_groups.
ServerHello	label41	8446	4.2.9	ServerHello MUST NOT send key_share when psk_ke is used.
ServerHello	label42	8446	4.1.3	legacy_session_id_echo MUST match the ClientHello session id.
ServerHello	label43	8446	4.1.3	legacy_compression_method MUST be null for TLS 1.3 ServerHello.
ServerHello	label44	8446	4.2.9	The server MUST NOT send psk_key_exchange_modes.
ServerHello	label45	8446	4.2.1	ServerHello MUST contain supported_versions.
ServerHello	label46	8446	4.2.9	psk_dhe_ke requires a server key_share.
ServerHello	label47	8446	4.2.11	selected_identity must be in the offered PSK range.
ServerHello	label48	8446	4.2.8	ServerHello must contain key_share unless PSK-only mode applies.
ServerHello	label49	8446	4.2.11	If PSK was offered, selected PSK extension must be present.
ServerHello	label50	8446	4 and 6	ServerHello syntax lengths must parse correctly.
ServerHello	label51	8446	4	The expected handshake message is ServerHello.
EncryptedExtensions	label52	8446	4.3.1	EncryptedExtensions MUST NOT contain forbidden extensions.
EncryptedExtensions	label53	8446	4 and 6	EncryptedExtensions syntax lengths must parse correctly.
EncryptedExtensions	label54	8446	4	The expected handshake message is EncryptedExtensions.
EncryptedExtensions	label55	8446	5	EncryptedExtensions is read under application-data record protection.
CertificateRequest	label56	8446	4.3.2	CertificateRequest MUST include signature_algorithms and only allowed extensions.
CertificateRequest	label57	8446	4.2	There MUST NOT be duplicate CertificateRequest extensions.
CertificateRequest	label58	8446	4 and 6	CertificateRequest syntax lengths must parse correctly.
CertificateRequest	label59	8446	4	CertificateRequest or Certificate may be expected in this state.
CertificateRequest	label60	8446	5	CertificateRequest is read under application-data record protection.
Certificate	label61	8446	4.4.2	Server certificate_list MUST be non-empty.
Certificate	label62	8446	4.4.2.2	Certificates MUST NOT use MD5 or SHA-1 signatures.
Certificate	label63	8446	4.4.2.2	Certificate public key/signature algorithms must be acceptable.
Certificate	label64	8446	4.4.2.2	Certificate signatures must verify.
Certificate	label65	8446	4 and 6	Certificate syntax lengths must parse correctly.
Certificate	label66	8446	4	Certificate handshake type must be valid for the current state.
Certificate	label67	8446	5	Certificate is read under application-data record protection.
Certificate	label68	8446	4.4.2.2	Client certificate signatures MUST NOT use MD5 or SHA-1.
Certificate	label69	8446	4.4.2.2	Client certificates must use acceptable signature algorithms.
Certificate	label70	8446	4.4.2.2	Client certificate signatures must verify.
Certificate	label71	8446	4.4.2	Client CertificateRequest context must match.
Certificate	label72	8446	4.4.2	Client certificate_list must satisfy local client-certificate policy.
Certificate	label73	8446	4 and 6	Certificate syntax lengths must parse correctly.
Certificate	label74	8446	4	The expected handshake message is Certificate.
Certificate	label75	8446	5	Client Certificate is read under application-data record protection.
CertificateVerify	label76	8446	4.4.3	CertificateVerify signature algorithm MUST NOT use SHA-1.
CertificateVerify	label77	8446	4.4.3	CertificateVerify signature algorithm must be offered.
CertificateVerify	label78	8446	4.4.3	CertificateVerify signature must verify against the transcript.
CertificateVerify	label79	8446	4 and 6	CertificateVerify syntax lengths must parse correctly.
CertificateVerify	label80	8446	4	The expected handshake message is CertificateVerify.
CertificateVerify	label81	8446	5	CertificateVerify is read under application-data record protection.
Finished	label82	8446	4.4.4	Finished verify_data must be correct.
Finished	label83	8446	4 and 6	Finished syntax lengths must parse correctly.
Finished	label84	8446	4	The expected handshake message is Finished.
Finished	label85	8446	5	Finished is read under application-data record protection.
CertificateRequest	label86	8446	4.6.2	Post-handshake CertificateRequest requires prior post_handshake_auth support.
KeyUpdate	label87	8446	4.6.3	KeyUpdate request_update value must be known.
KeyUpdate	label88	8446	4 and 6	KeyUpdate syntax lengths must parse correctly.
KeyUpdate	label89	8446	4	The expected handshake message is KeyUpdate.
ClientHello	label90	5246	7.4.1.2	ClientHello.client_version MUST be a supported TLS version.
ClientHello	label91	5246	7.4.1.2	compression_methods MUST contain CompressionMethod.null.
ClientHello	label92	5246	7.4.1.2	The server MUST select an acceptable cipher suite.
ClientHello	label93	5246	7.4.1.2	The server MUST select an acceptable compression method.
ClientHello	label94	5246	7.4.1.4	There MUST NOT be more than one extension of the same type.
ClientHello	label95	5246	7.4.1.4.1	The anonymous signature value MUST NOT appear in signature_algorithms.
ClientHello	label96	5246	7.4.4	Certificate authentication requires signature_algorithms in this model.
ClientHello	label97	5246	7.4.1.4	ClientHello extensions must be extensions supported by this TLS 1.2 model.
ClientHello	label98	5246	7.4.1.4.1	Signature algorithms must overlap with server-supported algorithms.
ClientHello	label99	5746	3.4	Client renegotiation_info MUST match previous verify_data.
ClientHello	label100	5246	7.4 and 6	ClientHello syntax lengths must parse correctly.
ClientHello	label101	5246	7.4	The expected handshake message is ClientHello.
ClientHello	label102	5246	6.2.1	ClientHello is carried in a handshake record.
ServerHello	label103	5246	7.4.1.3	ServerHello.server_version MUST be TLS 1.2 in this model.
ServerHello	label104	5246	7.4.1.3	The cipher suite selected by the server MUST be offered by the client.

TABLE XIII
CVE CORPUS AND COVERAGE SUMMARY

Metric	**Count**
Official CVE corpus, 2020–June 2026	129
RFC 5246/8446-related CVEs	34
Non-RFC 5246/8446 CVEs	95
Covered RFC 5246/8446-related CVEs	17/34 (50.0%)
Not covered in this evaluation	17

Message Type	Label ID	RFC	Section	Description
ServerHello	label105	5246	7.4.1.3	The compression method selected by the server MUST be offered by the client.
ServerHello	label106	5246	7.4.1.4	ServerHello MUST NOT contain unsolicited extensions, except renegotiation_info.
ServerHello	label107	5246	7.4.1.4.1	Servers MUST NOT send supported_signature_algorithms.
ServerHello	label108	4492	5.1	Servers MUST NOT negotiate supported_groups in ServerHello in this model.
ServerHello	label109	5246	7.4.1.4	There MUST NOT be duplicate ServerHello extensions.
ServerHello	label110	5746	3.6	Server renegotiation_info MUST match previous verify_data.
ServerHello	label111	5246	7.4 and 6	ServerHello syntax lengths must parse correctly.
ServerHello	label112	5246	7.4	The expected handshake message is ServerHello.
ServerHello	label113	5246	6.2.1	ServerHello is carried in a handshake record.
Certificate	label114	5246	7.4.2	Server certificates MUST be appropriate for the negotiated cipher suite.
Certificate	label115	5246	7.4.2	Server certificate chains must be issued by an accepted authority in this model.
Certificate	label116	5246	7.4.2	Server certificate signatures must verify.
Certificate	label117	5246	7.4.2	If signature_algorithms was sent, certificate signatures must use an offered pair.
Certificate	label118	5246	7.4.2	A non-anonymous server MUST send a non-empty certificate_list.
Certificate	label119	5246	7.4 and 6	Certificate syntax lengths must parse correctly.
Certificate	label120	5246	7.4	The expected handshake message is Certificate.
Certificate	label121	5246	6.2.1	Certificate is carried in a handshake record.
Certificate	label122	5246	7.4.6	Client certificates MUST be appropriate for the negotiated cipher suite.
Certificate	label123	5246	7.4.6	Client certificates should be issued by one of the requested authorities.
Certificate	label124	5246	7.4.6	Client certificate signatures must verify.
Certificate	label125	5246	7.4.6	Client certificates must use a requested signature algorithm.
Certificate	label126	5246	7.4.6	This model requires a non-empty client certificate_list when client auth is requested.
Certificate	label127	5246	7.4 and 6	Certificate syntax lengths must parse correctly.
Certificate	label128	5246	7.4	The expected handshake message is Certificate.
Certificate	label129	5246	6.2.1	Certificate is carried in a handshake record.
ClientKeyExchange	label130	5246	7.4.7	ClientKeyExchange syntax lengths must parse correctly.
ClientKeyExchange	label131	5246	7.4	The expected handshake message is ClientKeyExchange.
ClientKeyExchange	label132	5246	6.2.1	ClientKeyExchange is carried in a handshake record.
CertificateVerify	label133	5246	7.4.8	CertificateVerify signatures must verify over the handshake transcript.
CertificateVerify	label134	5246	7.4.8	CertificateVerify signature algorithm must be requested.
CertificateVerify	label135	5246	7.4 and 6	CertificateVerify syntax lengths must parse correctly.
CertificateVerify	label136	5246	7.4	The expected handshake message is CertificateVerify.
CertificateVerify	label137	5246	6.2.1	CertificateVerify is carried in a handshake record.
ServerKeyExchange	label138	5246	7.4.3	ServerKeyExchange signatures must verify when present.
ServerKeyExchange	label139	5246	7.4.3	Ephemeral ECDH parameters must use an accepted group.
ServerKeyExchange	label140	5246	7.4 and 6	ServerKeyExchange syntax lengths must parse correctly.
ServerKeyExchange	label141	5246	7.4	The expected handshake message is ServerKeyExchange.
ServerKeyExchange	label142	5246	6.2.1	ServerKeyExchange is carried in a handshake record.
CertificateRequest	label143	5246	7.4 and 6	CertificateRequest syntax lengths must parse correctly.
CertificateRequest	label144	5246	6.2.1	CertificateRequest is carried in a handshake record.
CertificateRequest	label145	5246	7.4	CertificateRequest or ServerHelloDone may be expected at this point.
ServerHelloDone	label146	5246	7.4 and 6	ServerHelloDone syntax lengths must parse correctly.
ServerHelloDone	label147	5246	6.2.1	ServerHelloDone is carried in a handshake record.
ServerHelloDone	label148	5246	7.4	The expected handshake message is ServerHelloDone.
ChangeCipherSpec	label149	5246	7.1	ChangeCipherSpec syntax lengths must parse correctly.
ChangeCipherSpec	label150	5246	6.2.1	ChangeCipherSpec is carried in a change_cipher_spec record.
Finished	label151	5246	7.4.9	Finished verify_data must match the handshake transcript and master secret.
Finished	label152	5246	7.4 and 6	Finished syntax lengths must parse correctly.
Finished	label153	5246	7.4	The expected handshake message is Finished.
Finished	label154	5246	6.2.1	Finished is carried in a handshake record.

APPENDIX K
DETAILED RQ4 CVE REPRODUCTION RESULTS

TABLE XIV
RQ4: DEVIATION-GUIDED REPRODUCTION OF PREVIOUSLY KNOWN CVES

CVE	Target	Outcome reproduced	Pattern	Test Fails
CVE-2025-6395 [23]	GnuTLS 3.7.9	Memory crash	P2	1
CVE-2023-3724 [24]	WolfSSL 5.6.0	Key compromise	P2	1
CVE-2022-39173 [25]	WolfSSL 5.5.0	Memory crash	P3	TD
CVE-2022-25638 [26]	WolfSSL 5.1.0	Client impersonation	P3	TD
CVE-2022-25640 [27]	WolfSSL 5.1.0	Client impersonation	P5	1
CVE-2021-44718 [28]	WolfSSL 5.0.0	Infinite loop	P3	TD
CVE-2021-3336 [29]	WolfSSL 4.6.0	Server impersonation	P3	TD
CVE-2020-24613 [30]	WolfSSL 4.4.0	Server impersonation	P5	1
CVE-2026-25834 [31]	MbedTLS 4.0.0	Downgrade attack	P3	TD